

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16605-126888

Autonómne riadenie malého vozidla s detekciou prekážky

Bakalárska práca

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16605-126888

Autonómne riadenie malého vozidla s detekciou prekážky

Bakalárska práca

Študijný program:	aplikovaná informatika
Študijný odbor:	informatika
Školiace pracovisko:	Ústav informatiky a matematiky
Školiteľ:	Ing. Štefan Bucz, PhD.



ZADANIE BAKALÁRSKEJ PRÁCE

Autor práce:	Rudolf Tisoň
Študijný program:	aplikovaná informatika
Študijný odbor:	informatika
Evidenčné číslo:	FEI-16605-126888
ID študenta:	126888
Vedúci práce:	Ing. Štefan Bucz, PhD.
Vedúci pracoviska:	doc. Ing. Milan Vojvoda, PhD.

Názov práce:	Autonómne riadenie malého vozidla s detekciou prekážky
--------------	---

Jazyk, v ktorom sa práca vypracuje:	slovenský jazyk
-------------------------------------	-----------------

Špecifikácia zadania:	Cieľom bakalárskej práce je návrh a implementácia algoritmu autonómneho riadenia malého vozidla s detekciou prekážky pred vozidlom. Implementácia navrhnutého algoritmu je realizovaná na platforme Arduino s využitím funkcií OpenCV.
-----------------------	--

Termín odovzdania práce:	06. 06. 2026
--------------------------	--------------

Podakovanie

Na tomto mieste by som rád poďakoval svojmu vedúcemu práce Ing. Štefanovi Buczovi, PhD. za cenné rady a čas, ktorý mi venoval. Jeho odborná pomoc a podpora mali významný vplyv na výsledok mojej bakalárskej práce a boli značnou oporou pri jej písaní.

Abstrakt

Cieľom bakalárskej práce je návrh a implementácia algoritmu autonómneho riadenia malého vozidla s detekciou prekážky pred vozidlom. Implementácia navrhnutého algoritmu je realizovaná na platforme Arduino s využitím funkcií OpenCV.

Kľúčové slová

Arduino, OpenCV, Python, detekcia prekážky, autonómne vozidlo

Abstract

The aim of the bachelor's thesis is to design and implement an algorithm for autonomous control of a small vehicle with obstacle detection in front of the vehicle. The implementation of the proposed algorithm is carried out on the Arduino platform using OpenCV functions.

Keywords

Arduino, OpenCV, Python, obstacle detection, autonomous vehicle

Obsah

Úvod	11
1 Autonómne vozidlá	12
1.1 Čo je to autonómne vozidlo?	12
1.2 Úrovne autonómnych vozidiel	12
1.2.1 Úroveň 0	13
1.2.2 Úroveň 1	13
1.2.3 Úroveň 2	13
1.2.4 Úroveň 3	13
1.2.5 Úroveň 4	13
1.2.6 Úroveň 5	13
2 Ciele práce	14
3 Analýza súčasného stavu	15
3.1 Aktuálny stav technológie	15
3.1.1 Umelá inteligencia	15
3.1.2 Senzory	16
3.2 Použitie autonómnych vozidiel	17
3.2.1 Autonómne taxi služby	17
3.2.2 Autonomizácia donášky jedla	17
3.3 Zhrnutie poznatkov	18
4 Komponenty systému	20
4.1 Arduino Nano ESP32 mikrokontrolér	20
4.2 ESP32-CAM + OV2640 kamera	21
4.3 HC-SR04 Ultrazvukový senzor	22
4.4 DRV8833 driver pre motorčeky	22
4.5 Ďalšie elektrické komponenty	23
4.6 Server	24
5 Softvér	25
5.1 C++	25
5.2 Python	25
5.3 OpenCV	25
5.4 NumPy	26
5.5 CNN	26

5.5.1	Ultralytics YOLOv8	27
6	Návrh a implementácia	28
6.1	Detekcia vozovky	28
6.1.1	Region of interest	28
6.1.2	Hľadanie tmavých regiónov	29
6.1.3	Hľadanie kontúr vozovky	31
6.2	Detekcia prekážok	31
6.2.1	Ultrazvukový senzor	31
6.2.2	Ultralytics YOLOv8	33
6.3	Interpretácia dát	33
6.4	Návrh komunikácie komponentov	39
6.4.1	Komunikácia kamery a serveru	39
6.4.2	Komunikácia serveru a mikrokontroléra	41
6.4.3	Komunikácia mikrokontroléra s perifériami	41
6.5	Návrh rámu a rozloženie komponentov	43
6.6	Montovanie vozidla a nahratie softvéru	43
7	Metodika testovania	47
7.1	Autonómne riadenie	47
7.2	Detekcia prekážky	48
8	Výsledky	51
8.1	Testovanie detekcie vozovky	51
8.2	Testovanie ultrazvukového senzora	52
8.3	Testovanie konvolučnej neurónovej siete	53
	Záver	56
	Literatúra	57
	Použitie nástrojov umelej inteligencie	59
A	Výpis dlhého kódu	60

Zoznam obrázkov a tabuliek

Obrázok 1	Ako AV vnímajú prostredie	16
Obrázok 2	Arduino Nano ESP32	20
Obrázok 3	ESP32-CAM a OV2640	21
Obrázok 4	HC-SR04 Ultrazvukový senzor	22
Obrázok 5	DRV8833 driver pre motorčeky	23
Obrázok 6	Typická CNN	26
Obrázok 7	Obraz s použitím ROI	29
Obrázok 8	Hľadanie tmavého jazdného pruhu.	30
Obrázok 9	Diagram fungovania ultrazvukového senzora	32
Obrázok 10	Detekcia prekážky pomocou CNN.	35
Obrázok 11	Hľadanie stredu jazdného pruhu.	36
Obrázok 12	Sekvenčný diagram vyhodnotenia dát zo senzorov.	38
Obrázok 13	Diagram komunikácie komponentov	39
Obrázok 14	3D model vozidla	43
Obrázok 15	Integrované vývojové prostredie Arduino	44
Obrázok 16	Delič napätia	45
Obrázok 17	Skompletizované vozidlo (predok)	46
Obrázok 18	Skompletizované vozidlo (zadok)	46
Obrázok 19	Testovacia trať	48
Obrázok 20	Prekážky na testovanie ultrazvukového modulu.	49
Obrázok 21	Prekážky na testovanie CNN.	50
Tabuľka 1	Hodnoty: Detekcia jazdného pruhu	51
Tabuľka 2	Hodnoty : Ultrazvukový senzor – stred vozovky	52
Tabuľka 3	Hodnoty : Ultrazvukový senzor – mimo vozovky	52
Tabuľka 4	Hodnoty : Ultrazvukový senzor – pri vozovke	53
Tabuľka 5	Hodnoty : CNN – automobil	53
Tabuľka 6	Hodnoty : CNN – Nákladné auto	54
Tabuľka 7	Hodnoty : CNN – Bicykel	54
Tabuľka 8	Hodnoty : CNN – Človek	54
Tabuľka 9	Hodnoty : CNN – Pes	55

Zoznam značiek a skratiek

ADR	Autonómne doručovacie roboty
AI	Umelá inteligencia
API	Application programming interface
AV	Autonómne vozidlo
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DC	Direct Current
GPIO	General-purpose input/output
GPS	Globálny polohovací systém
GPU	Graphics Processing Unit
IDE	Integrated Development Enviroment
LiDAR	Light Detection and Ranging
RADAR	Radio Detection and Ranging
RAM	Random Access Memory
ROI	Region of interest
RPM	Revolutions per minute
YOLO	You Only Look Once

Zoznam výpisov kódov

1	Funkcia ROI	28
2	Definície maxima a minima tmavých regiónov	29
3	Maskovanie tmavých regiónov	30
4	Detekcia kontúr	31
5	Obsluha prerušenia ultrazvukového senzora	32
6	Výpočet vzdialenosti z merania ultrazvukového senzora	33
7	Príprava CNN na detekciu objektov	33
8	Rozhodovanie zastavenia pri detekcii pomocou CNN	34
9	Výpočet geometrického stredu kontúry	35
10	Výpočet hodnoty riadenia	36
11	Rozhodovanie na vozidle	37
12	Zapnutie Wi-Fi hotspotu	40
13	Pingovanie IP adresy kamerového modulu	40
14	Spustenie HTTP stream na ESP32-CAM	41
15	Posielanie UDP packetov	41
16	Vnútorňý cyklus Arduina	42

Úvod

Autonómne vozidlá sú v súčasnosti jedným z najrýchlejšie sa rozvíjajúcich technologických odvetví. Ide o formu mobility, ktorá je bezpečnejšia, efektívnejšia a pohodlnejšia než konvenčné spôsoby dopravy riadené človekom. Zavedenie autonómnych vozidiel vedie k presunu pracovnej sily od rutinných manuálnych úkonov smerom k činnostiam zameraným na monitorovanie, manažovanie a kontrolu systémov. Taktiež umožňujú ľuďom s fyzickým postihnutím samostatne operovať automobil. V súčasnosti sú vozidlá s čiastočnou autonómiou dostupné pre každého, kto túži po luxuse, ktorý tieto autá poskytujú, a má na to potrebné financie.

Autonómne vozidlá sa rozdeľujú do šiestich stupňov autonómie, od 0 (žiadna autonómia) až po 5 (úplná autonómia bez nutnosti zásahu človeka). Na nahradenie funkcií človeka využívajú senzory na vnímanie okolia a umelú inteligenciu na rozhodovanie. Technologické spoločnosti sa v tejto oblasti snažia o čo najefektívnejšie prepojenie týchto dvoch systémov.

Najväčšími bariérami pri integrácii autonómnych vozidiel do bežnej spoločnosti sú ich vysoká cena a neschopnosť navigovať v komplikovaných hraničných situáciách. Súčasný vývoj sa snaží tieto bariéry aktívne prekonávať.

Cielom bakalárskej práce je navrhnúť a implementovať riešenie autonómneho vozidla, ktoré je schopné detekovať prekážky a adekvátne na ne reagovať. Projekt realizujeme na platforme Arduino s využitím knižnice na počítačové videnie OpenCV. V rámci práce preskúmame aktuálne prístupy k autonómii a zvažíme životaschopnosť súčasných trendov. Výstupom bude praktická implementácia modelu vozidla a jeho testovanie v rôznych situáciách.

Výber témy bakalárskej práce vychádza z nášho záujmu o automatizáciu a inovácie v oblasti automobilov. Daná téma spája mnohé populárne technológie, ako je umelá inteligencia a automatizácia logistiky. Zároveň nás zaujala práca s mikrokontrolérmi a elektronikou. Práve kombináciou týchto oblastí sme sa pre túto tému nadchli.

1 Autonómne vozidlá

1.1 Čo je to autonómne vozidlo?

Autonómne vozidlo (AV), niekedy nazývané aj samoriadiace auto je dopravný prostriedok, ktorý, na rozdiel od tradičných vozidiel nevyžaduje vodiča. Autonómne vozidlá vykonávajú túto úlohu pomocou viacerých hardvérových a softvérových systémov. Hoci ani v roku 2026 nemá termín 'samoriadiace' jednotnú štandardnú definíciu, hlavným cieľom týchto technológií zostáva eliminácia ľudských chýb, zvýšenie efektivity premávky a sprístupnenie prepravy ľuďom, ktorí nie sú schopní šoférovať sami.

Na vnímanie okolia vozidla používame senzory detekcie objektov. Systém vnímania používa kamery, Radio Detection and Ranging (RADAR), Light Detection and Ranging (LiDAR) a ultrazvukové senzory, pričom každý z nich má špecifickú funkciu. Autonómne vozidlá často pracujú s čiastočne neznámym a dynamickým prostredím, preto si musia vystavať model svojho prostredia a zároveň sa v ňom lokalizovať. Tento proces je známy pod skratkou SLAM (súčasná lokalizácia a mapovanie).

Spočíva v spracovaní dát zo senzorov, máp a systéme globálneho polohovacieho systému GPS, vďaka čomu si zariadenie vytvorí model prostredia a určí v ňom svoju polohu v reálnom čase [1].

Tretí systém je zodpovedný za plánovanie a rozhodovanie. Na základe dát z predchádzajúcej vrstvy vozidlo vyhodnocuje a reaguje na vývoj dopravnej situácii. Táto vrstva má mnoho funkcií od plánovania trasy po vyhýbanie sa prekážkam. Najčastejšie sa používajú modely strojového učenia, vrátane neurónových sietí [1].

Posledným kľúčovým článkom je konanie a ovládanie. Systém ovládania vozidla premení abstraktné požiadavky predchádzajúcej vrstvy na fyzické pohyby auta. Zodpovedá za smerové riadenie, zrýchľovanie a brzdenie.

Pokročilé algoritmy zároveň zabezpečujú, aby pohyb bol plynulý, stabilný a rešpektoval limitácie podvozku. Vďaka integrácii týchto systémov dokáže autonómne vozidlo navigovať v komplexných dopravných situáciách s minimálnou až nulovou potrebou ľudského zásahu [1].

1.2 Úrovne autonómnych vozidiel

Úrovne autonómnych vozidiel sú definované organizáciou SAE International v štandarde SAEJ3016. Tento dokument delí automatizáciu riadenia do šiestich stupňov (0-5) podľa toho, akú časť jazdy zabezpečuje systém a akú úlohu ešte stále zohráva vodič. O samoriadiacich autách sa často hovorí, ako o jednej kategórii, no v praxi ide o postupný

vývoj od minimálnej podpory riadenia až po úplnú autonómiu vozidla bez zásahu človeka [2].

1.2.1 Úroveň 0

Žiadna automatizácia Na tejto úrovni všetky činnosti vykonáva ľudský vodič. Vozidlo môže mať bezpečnostné systémy, ako napríklad upozornenie opustenia jazdného pruhu, no samotné riadenie je plne v rukách človeka [2].

1.2.2 Úroveň 1

Asistované riadenie Systém pomáha s postranným alebo pozdĺžnym riadením vozidla ale nie v obidvoch prípadoch naraz. Sem patria asistencie, ako udržiavanie rýchlosti (tempomat) alebo mierne korigovanie smeru. Vodič však musí neustále sledovať situáciu na vozovke a mať ruky na volante [2].

1.2.3 Úroveň 2

Čiastočná automatizácia Vozidlo dokáže súčasne riadiť smer aj rýchlosť, stále však platí, že vodič nepretržite dohliada na jazdu a je pripravený zasiahnuť v prípade núdze. Systém vykonáva viac úloh riadenia naraz, no zodpovednosť upadá na človeka [2].

1.2.4 Úroveň 3

Podmienená automatizácia Jazda v špecifickej situácii je automatizovaná a vodič nemusí aktívne sledovať okolie. Ak však systém vyhodnotí, že situáciu nezvládne, kontrola vozidla sa vráti vodičovi. Vodič musí byť pripravený prevziať kontrolu nad vozidlom [2].

1.2.5 Úroveň 4

Vysoká automatizácia Na tejto úrovni je vozidlo schopné zvládnuť jazdu úplne samo v rámci definovaných prostredí alebo podmienok. V prípade, že sa vozidlo dostane mimo týchto podmienok, dokáže bezpečne zastaviť. Prítomnosť ľudského šoféra nie je nutná [2].

1.2.6 Úroveň 5

Plná automatizácia Najvyšší stupeň automatizácie, vozidlo dokáže jazdiť samostatne za všetkých podmienok, v ktorých dokáže jazdiť človek. Neexistuje potreba vodiča. Vozidlo zastaví v prípade zlyhania určitých modulov riadenia [2].

2 Ciele práce

V tejto kapitole si stanovíme hlavný cieľ bakalárskej práce a taktiež čiastkové ciele, ktoré slúžia, ako milníky pri praktickom spracovaní témy práce.

Hlavný cieľ bakalárskej práce je návrh a implementácia algoritmu pre autonómne riadenie malého robotického vozidla, ktoré dokáže v reálnom čase detegovať prekážky v jazdnej dráhe a následne na ne reagovať.

Praktická časť práce sa zameriava na implementáciu riešenia na hardvérovej platforme Arduino so súčasnými metódami počítačového videnia pomocou knižnice OpenCV.

Čiastkové ciele

1. **Analýza problematiky:** Teoretické spracovanie aktuálnej problematiky v oblasti autonómnych vozidiel. Zhodnotenie súčasnej a budúcej implementácie jednotlivých systémov použitých v AV. Prehľad konkrétneho využitia senzorových systémov a umelej inteligencie na detekciu prekážok.
2. **Návrh technologického riešenia:** Výber optimálnej kombinácie hardvérových a softvérových komponentov v rámci obmedzenia bakalárskej práce. Zvolenie riadiacej jednotky Arduino, typ a počet senzorov, programovací jazyk riešenia a mechanické prvky vozidla. Výber metód detekcie prekážky a jazdného pruhu pomocou počítačového videnia. Fyzický návrh vzhľadu vozidla so zameraním na použité materiály a rozloženie komponentov.
3. **Algoritmizácia detekcie:** Zvolenie riešení v prostredí OpenCV zameraných na spracovanie obrazu z kamery na identifikáciu prekážok a jazdného pruhu v zornom poli vozidla.
4. **Prepojenie komponentov:** Návrh metód komunikácie jednotlivých komponentov. Prepojenie vrstiev riadenia autonómneho vozidla od vnímania po konanie. Interpretácia výstupov na vstupy pre jednotlivé systémy. Návrh fyzického prepojenie častí systému pomocou komunikačného média.
5. **Zostavenie riešenia:** Implementácia zvolených návrhov. Naprogramovanie riešenia s použitím knižnice OpenCV. Zostavenie rámu vozidla a následná montáž.
6. **Testovanie a validácia:** Navrhnutie metodiky testovania a následné overenie funkčnosti vozidla v rozličných situáciách. Otestovanie detekcie rôznych prekážok.

3 Analýza súčasného stavu

3.1 Aktuálny stav technológie

V posledných rokoch sme zaznamenali nárast vydání automobilov tretej úrovne autonómnosti, autonómne taxi služby sa stávajú realizovateľným spôsobom dopravy v mestách a dostali sme prvé ukážky kamiónov bez nutnosti ľudského vodiča [3][4].

Vďaka vývoju v oblasti umelej inteligencie a zvýšeniu výkonu výpočtových platforiem sme zaregistrovali značný pokrok v technológiách autonómnych vozidiel. Výrobcovia sa snažia zlepšovať kombinácie RADARových, LiDARových a kamerových dát tak, aby boli robustnejšie voči dopravným a poveternostným podmienkam. Pokračuje snaha v návrhu scenárov pre hraničné situácie, čím sa zlepší percepčná schopnosť autonómnych systémov a zrýchli ich nasadenie [5][6].

Autonómne vozidla zaznamenali značnú investíciu ale aj kontrolu zo strany štátnych a nadnárodných organizácií. Spoločnosti, ktoré poskytujú služby samoriadiacich vozidiel rozširujú svoju dostupnosť, znižujú počet senzorov a náklady na hardvér. Hoci technologický pokrok napreduje, integrácia do infraštruktúry mesta zostáva podstatnou výzvou pre adopciu AV do dopravy. Štandard SAE International preto opisuje plán prechodu od asistencie šoférovanie po plne autonómnu prevádzku [7][8].

3.1.1 Umelá inteligencia

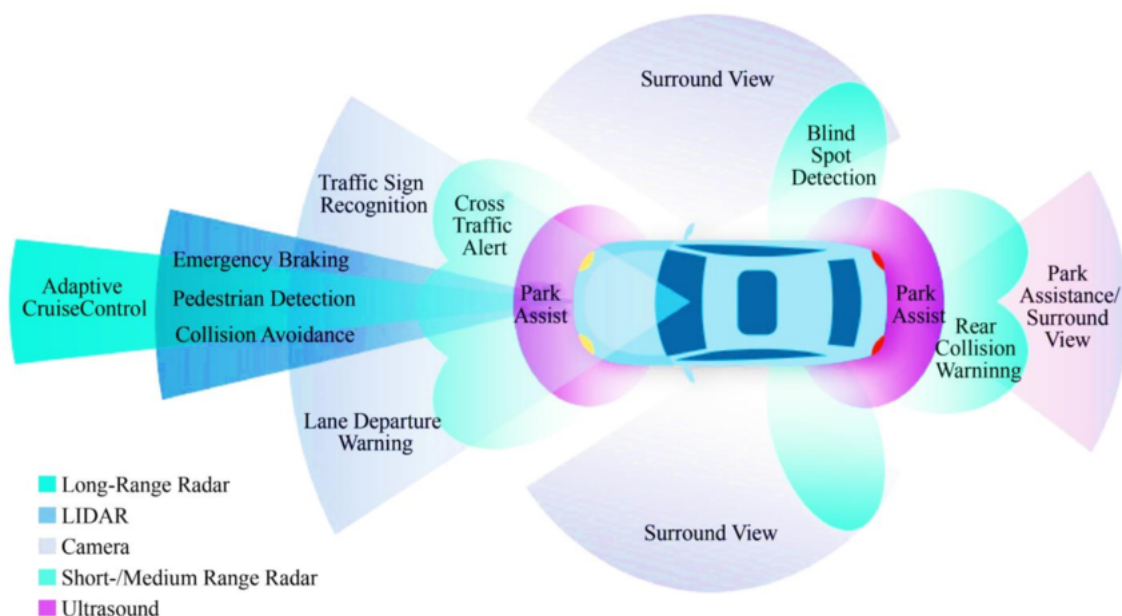
Umelá inteligencia (AI) vykonáva dôležitú funkciu v percepčnom, rozhodovacom a plánovacom systéme autonómneho vozidla, čím nahrádza rolu ľudského vodiča. Moderné systémy používajú strojové učenie (machine learning) na detekciu hazardov na ceste, klasifikáciu dopravných značiek a sledovanie a predikciu pohybu chodcov a ostatných vozidiel [9].

V súčasnosti sa autonómne modely trénujú na obrovskom množstve anotovaných obrázkov, videí, simulácií a skutočných jazdných dát, aby v praxi zvládli širokú škálu scenárov. Hlboké učenie (deep learning) a učenie posilňovaním (reinforcement learning) sa používajú v novodobých autonómnych vozidlách. Deep learning, podskupina strojového učenia, pracuje na princípe viacvrstvových neurónových sietí, čím sa fungovaním podobá ľudskému mozgu. Reinforcement learning mení správanie modelu využitím systému odmien a penalizácií [10].

Napriek značnej komplexnosti zostáva riešenie hraničných prípadov (edge cases) zásadnou výzvou pre dnešné systémy. Potreba veľkého množstva tréningových dát a vysoká hranica bezpečnosti a spoľahlivosti výrazne spomaľuje nasadenie vozidiel do reálnej premávky [6].

3.1.2 Senzory

Senzory sú spôsob akým autonómne vozidlá vnímajú svoje okolie. Ako je možné vidieť na obrázku 1, v súčasnosti sa v autách využíva kombinácia kamier, LiDARových, RADARových a ultrazvukových senzorov [11].



Obr. 1: Ako AV vnímajú prostredie
[9], licencované pod CC BY 4.0.

Kamery predstavujú najrozšírenejší typ senzorov; sú nákladovo efektívne, no citlivé na zlé osvetlenie a počasie. Používajú sa na detekciu jazdných pruhov, rozpoznávanie dopravných značiek a identifikáciu chodcov a vozidiel. RADAR využíva elektromagnetické vlny na meranie vzdialenosti a rýchlosti. Používa sa najmä pre detekciu vzdialených objektov aj v nepriaznivých poveternostných podmienkach (dážď, hmla, sneh). LiDAR používa laserové impulzy na vytvorenie detailného trojrozmerného modelu okolia, no okrem vysokých nákladov je aj viac náchylný na zlé počasie. Ultrazvukové senzory majú využitie v detekcii objektov na krátke vzdialenosti, napríklad pri parkovacích manévroch. Na rozdiel od fotoelektrických senzorov (napr. RADAR), nie sú ovplyvnené farbou alebo odrazivosťou cieľa, pretože používajú zvuk a nie svetlo[1] [11].

Súčasný vývoj v oblasti senzorov sa zameriava primárne na zvyšovanie presnosti meranie, optimalizovanie počtu potrebných senzorov a výrazné znižovanie ich ceny. Výrobcovia taktiež zefektívňujú kombinovanie dát viacerých senzorov, tzv. senzorovú fúziu, čím zvyšujú presnosť a spoľahlivosť systému [8][9].

3.2 Použitie autonómnych vozidiel

Autonómne vozidlá ponúkajú široké spektrum využitia, pričom ich možnosti sa v posledných rokoch rozširujú. K najpozoruhodnejším oblastiam ich využitia patrí doprava. Hlavným cieľom je zvýšiť bezpečnosť a plynulosť premávky. Ďalšie využitie je v logistike tovaru, kde nahrádzajú rutinnú manuálnu prácu tradične vykonávanú ľuďmi.

3.2.1 Autonómne taxi služby

Autonómne taxi služby, často nazývané robo-taxíky predstavujú jednu z najpokročilejších foriem využitia autonómnych služieb. Proces od objednania, naplánovania trasy po prepravy pasažierov je plne automatizovaný. Mnohí výrobcovia automobilov, technologické spoločnosti a štátne organizácie investujú zdroje do vývoju robo-taxíkov. Na popredí je firma Waymo, ktorá je dcérskou spoločnosťou Alphabet. Ich automobily, ktoré spĺňajú štvrtú úroveň autonómie, využívajú LiDAR, RADAR a kamery na vytvorenie trojrozmerného obrazu prostredia. S každou ďalšou generáciou vozidiel redukujú počet senzorov, čím znižujú náklady na výrobu a zlepšujú škálovateľnosť projektu [8].

Alternatívny prístup zvolila spoločnosť Tesla, so službou Robotaxi a systémom FSD (Full Self-Driving), forma autonómneho riadenia tretieho stupňa. Používa kamerový systém s interpretáciou dát pomocou umelej inteligencie namiesto finančne nákladových LiDAR senzorov. Ľudský operátor dohliada nad jazdou a je pripravený prevziať kontrolu nad riadením, no v roku 2026 začala firma zavádzať vozidlá bez dozoru do ich flotily [12].

Ďalším dôležitým hráčom je spoločnosť Nvidia, s modelom Alamayo 1. Tento open source model uvažovania je trénovaný na riešenie komplexných a kritických dopravných situácií. Toto voľné riešenie má za úlohu urýchliť adopciu štvrtej a piatej úrovne AV do bežnej premávky [6].

3.2.2 Autonomizácia donášky jedla

Autonómne doručovacie roboty (ADR) sa používajú v súčasnosti v univerzitných kampusoch a mestách na donášku jedla. Poskytujú takzvanú last-mile delivery (dodanie na poslednú míľu) a fungujú na štvrtej úrovni autonómie. Primárne využívané na chodníkoch alebo cyklochodníkoch, doručovacie roboty sú schopné prekonávať križovatky, vyhýbať sa prekážkam a doručovať tovar na vzdialenosť pár kilometrov [13].

ADR používajú zväčša kombináciu kamier a počítačového videnia na navigovanie, týmto spôsobom zabezpečujú nižšie náklady na výrobu, s väčšou mierou škálovateľnosti. Niektorí výrobcovia sa vyhýbajú používaniu GPS, pretože neposkytuje dostatočnú mieru presnosti [14].

Autonómna donáška jedla zaznamenala výrazný rozvoj počas pandémie COVID-19. Mnohé spoločnosti poskytujúce donášku jedla, ako napríklad Uber Eats a DoorDash, ale aj logistické spoločnosti, ako Amazon, Walmart a Google začali investovať do vývoja ADR. Jedným z priekopníkov v tejto oblasti je spoločnosť Starship Technologies, ktorá výrazne rozšírila svoju flotilu práve počas pandémie. Zatiaľ čo donáška pomocou pozemných robotov je bežná, mnohé spoločnosti investujú aj do vývoja doručovania pomocou autonómnych dronov [14]. Značným problémom ADR je ich krátky dosah, pomalá rýchlosť a obmedzená kapacita užitočného zaťaženia. Mnohé spoločnosti investujú do hybridných modelov donášky kombináciou konvenčných donáškových vozidiel a doručovacích robotov [13].

V závislosti od charakteristík systému možno existujúci model hybridného doručovania klasifikovať, ako nasledujúce tri kategórie:

- **Dvojúrovňový model:** Tento model využíva konvenčné vozidlá (kamióny alebo dodávky) na hromadnú prepravu zásielok z centrálného depa do siete menších lokálnych skladísk. Z týchto miest vykonávajú ADR donášku na poslednú míľu priamo zákazníkovi [13].
- **Mothership model:** V tomto modeli slúži štandardné donáškové vozidlo ako materská loď, ktorá je špeciálne upravená na prepravu flotily ADR. Autonómne vozidlá sa vypúšťajú z materskej lode v bezprostrednej blízkosti cieľov donášky. Ide o jeden z najčastejšie skúmaných modelov v aktuálnej literatúre [13].
- **Model čaty** Autonómne vozidlá doručujú zásielky samostatne v oblastiach, ktoré sú schopné navigovať. Ak však musia prekonať zóny, ktoré sú pre roboty nedostupné, pripoja sa k manuálne riadenému vozidlu, ktoré funguje ako „vodca čaty“. Roboty nasledujú toto vozidlo, až kým nedorazia do zóny im vhodnej [13].

3.3 Zhrnutie poznatkov

Po analýze technologického vývoja autonómnych vozidiel je nevyhnutné kriticky zhodnotiť ich dopad na reálne dopravné prostredie. V nasledujúcej podkapitole si identifikujeme kľúčové trendy a vyvodíme poznatky, ktoré formujú súčasnú sféru autonómnej mobility.

Mnohé implementácie v priemysle autonómnych vozidiel sa posúvajú od asistovaných systémov úrovne 2 a 3, k plne autonómnej prevádzke v špecifických podmienkach úrovne 4. Plná autonómia úrovne 5 stále ostáva dlhodobým problémom v danej oblasti.

Počet senzorov sa v autonómnych vozidlách zdatne znižuje. Prechádza sa z ideológie „maximálnej redundancie“ s vysokým počtom senzorov na optimalizovaný

prístup. Firmy, ako Tesla nahrádzajú spoľahlivé, no ekonomicky nevýhodné LiDARové senzory za kombináciu kamier a AI.

Kontrastne, spoločnosti ako Waymo sa držia prístupu založeného na LiDARe, no snažia sa znížiť cenu a zvýšiť ich efektivitu. V súčasnosti ešte neexistuje konsenzus medzi technologickými spoločnosťami.

AI zohráva dôležitú rolu pri plánovaní a rozhodovaní systéme autonómnych vozidiel. Súčasný trend ukazuje na využívanie veľkých jazykových modelov, ktoré sú schopné interpretovať dáta zo senzorov a z nich priamo rozhodnúť o chode autonómneho vozidla.

Hraničné situácie zohrávajú najväčšiu bariéru ku masovému nasadeniu autonómnych vozidiel úrovne 5. Vývoj v oblasti umelej inteligencie sa zameriava práve na tieto edge cases. Z analýzy vyplýva, že vyriešením tohto problému, bude zavedenie AV v mestách zdatne urýchlené.

Technológia samotná nestačí pre adopciu autonómnych vozidiel do miest. Značným problémom je integrácia v mestskom prostredí a verejná akceptácia AV. Predpokladáme, že krajiny, ktoré budú investovať do integrácie autonómnych vozidiel, a ktoré majú progresívny názor na ich nasadenie, budú zaznamenávať najväčší nárast v tomto sektore.

Viacere z identifikovaných aspektov boli priamo reflektované v našom návrhu systému autonómneho vozidla s detekciou prekážok.

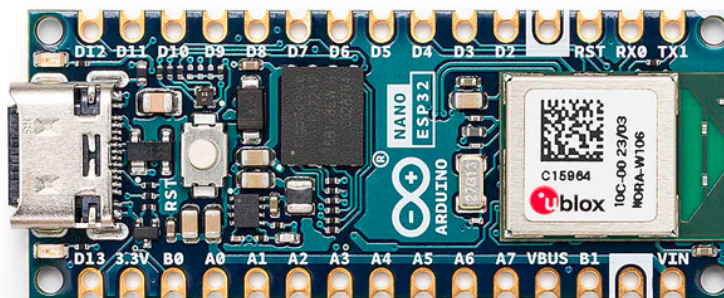
4 Komponenty systému

V tejto kapitole si preberieme použité komponenty celkového systému. Výber správnych súčastí v rámci limitácií bakalárskej práce je kľúčový pre získanie lepších výsledkov.

4.1 Arduino Nano ESP32 mikrokontrolér

Arduino je talianska open source hardvérová a softvérová firma, ktorá dizajnuje a vyrába mikrokontroléry na výrobu rôznych zariadení. Arduino Nano je rodina kompaktných a nákladovo efektívnych mikrokontrolérov. Nano ESP32 je osadený čipom ESP32-S3, ktorý integruje možnosti Wi-Fi a Bluetooth. Mikrokontrolér poskytuje správnu rovnováhu veľkosti a výkonu, s dostatočným počtom pinov na pripojenie periférií [15].

V našej práci používame Arduino Nano ESP32 ako riadiacu jednotku, ktorá získava vstupy od ostatných komponentov a podľa nich riadi mechanické časti vozidla [15].



Obr. 2: Arduino Nano ESP32

Arduino, licencované pod CC BY-SA 4.0, cez arduino.cc.

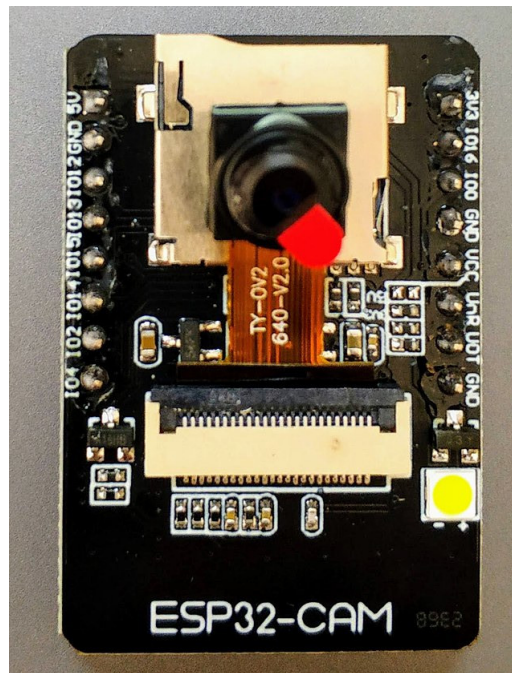
Technické parametre:

- Rozmery: 18 * 45mm
- Vstupné/výstupné napätie: 3.3V
- Počet GPIO pinov: 48
- Komunikačné rozhrania: UART, I2C, SPI

- Rýchlosť procesora: 240Mhz
- Veľkosť ROM: 384kB
- Veľkosť SRAM: 512kB
- Veľkosť RAM: 8MB

4.2 ESP32-CAM + OV2640 kamera

Vývojová doska ESP32-CAM integruje Wi-Fi a Bluetooth modul spolu s rozhraním na pripojenie 2MPx kamerový senzor OV2640. Táto kombinácia nám poskytuje oddelenú implementáciu prenosu videa, čím uvoľníme výpočtovú záťaž hlavného čipu. Senzor OV2640 disponuje s natívnym rozlíšením 1600 x 1200 pixelov a obrazovou frekvenciou 60 FPS, čo poskytuje dostatočnú výkonovú rezervu [16][17].



Obr. 3: ESP32-CAM a OV2640

Nowforever, licencované pod CC BY-SA 4.0, cez Wikimedia Commons.

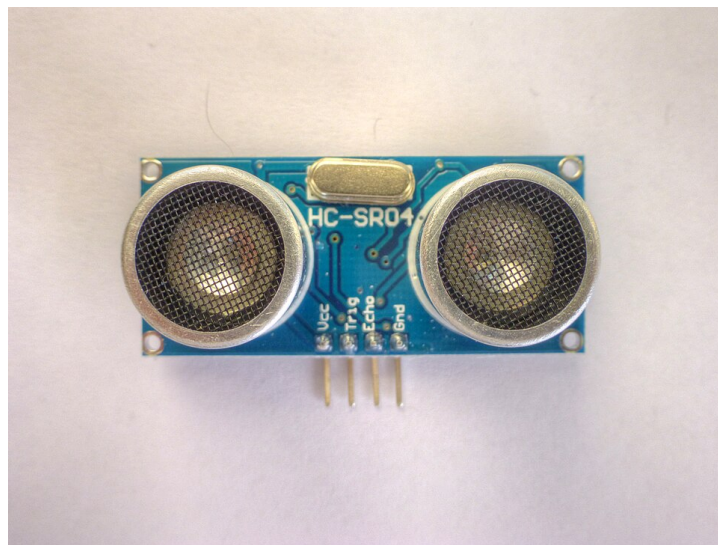
Technické parametre:

- Rozmery: 27 * 40mm
- Pracovné napätie: 2.2V - 3.6V
- Rozlíšenie kamery: 1600 x 1200
- Frame rate kamery: 60FPS (pri CIF formáte)

- Rýchlosť procesora: 240Mhz
- Veľkosť FLASH: 4MB
- Veľkosť PSRAM: 4MB

4.3 HC-SR04 Ultrazvukový senzor

HC-SR04 je nákladovo efektívne riešenie na detekciu prekážky pred vozidlom. Používame ho ako sekundárny systém na núdzové zastavenie vozidla v prípade, že kamerový systém nedetekuje prekážku [18].



Obr. 4: HC-SR04 Ultrazvukový senzor

Nevit Dilmen, licencované pod CC BY-SA 3.0, cez Wikimedia Commons.

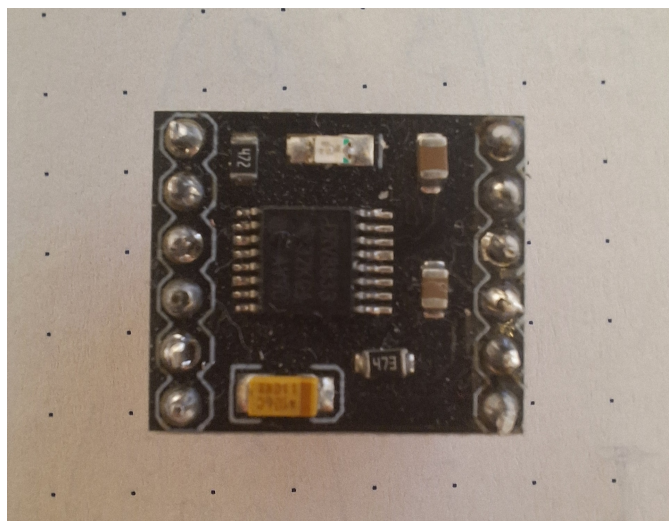
Technické parametre:

- Rozmery: 20 * 45 * 15mm
- Pracovné napätie: 5V
- Maximálna vzdialenosť: 4m
- Minimálna vzdialenosť: 2cm
- Merací úhol: 15°

4.4 DRV8833 driver pre motorčeky

DRV8833 sme použili na nezávislé riadenie dvoch DC motorov. Integrovaný obvod obsahuje dvojité H-mostík, ktorý umožňuje meniť polaritu napätia, a tým aj smer

otáčania motorov. Modul má štyri vstupné piny (dva pre každý motor), pomocou ktorých možno ovládať rýchlosť a smer otáčania [19].



Obr. 5: DRV8833 driver pre motorčeky

Technické parametre:

- Rozmery: 5 * 4 mm
- Logické napätie: 2.7V - 10.8V
- Drive napätie: 3V - 10V
- Drive prúd: 1.5A (MAX pre jeden mostík)
- Maximálny výkon: 15W

4.5 Ďalšie elektrické komponenty

V implementácii sme použili Li-Ion akumulátor s napätím 7.4V a kapacitou 3000mAh.

Pohon zabezpečujú dva DC motorčeky s prevodovkou na 200RPM pri 3V.

Kvôli ochrane komponentov vyžadujúce 5V napájanie sme do systému zapojili menič napätia (step-down buck converter). Použitý buck má vstup na dva vodiče a výstup pre dva USB A konektory.

Súčasťou zapojenia je taktiež trojpólový, dvojpolohový prepínač na vypnutie/zapnutie systému.

Prepojenie komponentov zabezpečuje nepájivé pole a káblíky.

4.6 Server

Vzhľadom na obmedzený výpočtový výkon mikrokontrolérov Arduino, ktoré nie sú schopné vykonávať komplexné výpočty neurónových sietí v reálnom čase, sme do systému zaradili centrálny server. Ten zabezpečuje spracovanie obrazu a koordináciu bezdrôtových periférií. V našej implementácii túto úlohu plní laptop, ktorý zároveň slúži ako prístupový bod pre Wi-Fi sieť.

5 Softvér

V nasledovnej časti sa oboznámime s použitým softvérom, ktorý nám umožňuje preklad požiadaviek systému na skutočné inštrukcie vykonateľné hardvérom.

5.1 C++

C++ je programovací jazyk vyššej úrovne na všeobecné použitie, ktorý umožňuje pracovať s prostriedkami nízkej úrovne. Bjarne Stroustrup vyvinul C++ v roku 1983 ako rozšírenie jazyka C [20].

Mikrokontroléry Arduino je možné programovať pomocou jazykov C/C++ vďaka štandardnému rozhraniu pre programovanie aplikácií (API) známemu ako Arduino Programming Language. Toto API obsahuje funkcie a knižnice na ovládanie hardvéru mikrokontroléra. Projekt Arduino taktiež poskytuje vývojové prostredie Arduino IDE, v ktorom je kompilácia, integrácia knižníc a nahrávanie kódu zjednodušené [15].

V našej implementácii používame C++ pre programovanie nízkoúrovňového mikrokontroléru Arduino Nano ESP32 a kamerového modulu ESP32-CAM.

5.2 Python

Python je interpretovaný, interaktívny programovací jazyk vysokej úrovne, ktorý vytvoril Guido van Rossum. Je dynamicky typovaný, čo znamená, že typ premennej sa určuje za behu programu. Na rozdiel od jazykov C/C++, využíva Garbage Collection, vďaka čomu programátor nemusí manuálne riešiť alokovanie a uvoľňovanie pamäte. Štruktúra kódu je definovaná odsadením, čím sa výrazne zvyšuje čitateľnosť kódu [21].

V našej implementácii používame Python na vytvorenie Wi-Fi servera, spracovanie dát z kamery a následnú interpretáciu výsledkov do podoby príkazov pre pohyb kolies.

5.3 OpenCV

OpenCV je knižnica programovacích funkcií určených na počítačové videnie v reálnom čase, ktorú pôvodne vyvinula spoločnosť Intel. Využíva sa v oblastiach detekcie objektov, tváří a gest, pri vizuálnej odometrii, vnímaní hĺbky či v augmentovanej realite. Knižnica OpenCV je napísaná v programovacom jazyku C++, no ponúka rozhrania pre integráciu v ďalších jazykoch, ako sú Python, Java alebo MATLAB [22][23].

V našej implementácii používame OpenCV na detekciu vozovky a analýzu jazdných pruhov.

5.4 NumPy

NumPy je knižnica pre programovací jazyk Python, ktorý pridáva podporu optimalizovaných polia, matice a matematické funkcie, ktoré s nimi pracuje [24].

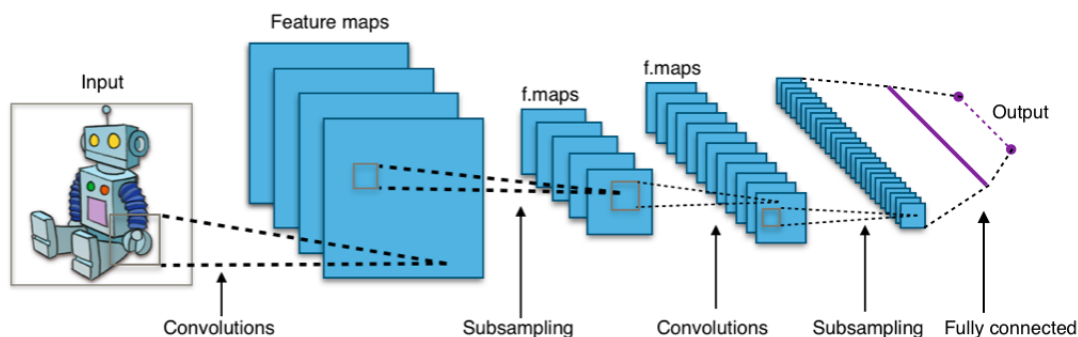
Python interpretuje inštrukcie riadok po riadku, čo spomaľuje celkový program. Knižnica NumPy urýchľuje tento proces tým, že vykonáva výpočtovo náročné operácie vo vysoko optimalizovanom, kompilovanom C kóde. Taktiež mnoho operácií uvoľňujú globálny zámok interpreteru, čím dovoľuje paralelné spúšťanie viacerých vlákien viacjadrového procesora. Horeuvedená knižnica OpenCV používa NumPy na ukladanie dát a operácie s nimi [23][24].

V našej implementácii používame knižnicu NumPy na prácu s vektormi a maticami pri transformácii obrazových dát.

5.5 CNN

Convolutional Neural Network (CNN), po slovensky konvolučná neurónová sieť, je typ doprednej neurónovej siete, ktorá používa optimalizáciu filtrov. Tento typ hlbkej neurónovej siete sa štandardne používa pri počítačovom videní a spracovaní obrazu [25].

Konvolučné neurónové siete sa skladajú z rôznych vrstiev. Hlavným stavebným prvkom je takzvaná konvolučná vrstva, ktorá obsahuje rôzne filtre – matice váh. Počas doprednej propagácie sa po celej výške a šírke vstupnej matice počíta skalárny súčin pre každý filter konvolučnej vrstvy. Výsledkom je dvojrozmerná aktivačná mapa daného filtra. CNN sa učia vytvárať filtre, ktoré sa aktivujú v prípade, že sa na vstupe nachádza požadovaná vlastnosť. Ďalšie vrstvy, ako združovacia (pooling) vrstva sa používa na zmenšenie rozmerov vstupnej matice, ReLU vrstva na odstránenie záporných hodnôt a plne prepojená (fully connected) vrstva na finálnu klasifikáciu. Obrázok 6 zobrazuje bežnú konvolučnú neurónovú sieť [25].



Obr. 6: Typická CNN

Aphex34, licencované pod CC BY-SA 4.0, cez Wikimedia Commons.

CNN dosahujú pri analýze obrazového vstupu výrazne nižšiu chybovosť a efektívnejší proces učenia v porovnaní s inými typmi neurónových sietí. V súčasnosti sa konvolučné neurónové siete používajú na detekciu prekážok v autonómnych vozidlách, no objavuje sa rastúci trend prechodu na veľké jazykové modely (LLM), ktoré kontrolujú všetky systémy vozidla [3][6][26].

5.5.1 Ultralytics YOLOv8

YOLOv8 je systém od spoločnosti Ultralytics určený na detekciu objektov v reálnom čase; funguje na základe konvolučných neurónových sietí. Metóda You Only Look Once (YOLO) vyžaduje iba jednu doprednú propagáciu (forward pass) na vytvorenie predpokladu, preto je ideálna pre aplikácie v reálnom čase [27][28].

Tento model je už predtrénovaný na rozsiahlych datasetoch, vďaka čomu je pripravený na okamžitú implementáciu bez nutnosti zložitej konfigurácie. V našej implementácii ho využívame na detekciu prekážky v dráhe vozidla.

6 Návrh a implementácia

Vhodný návrh architektúry je kľúčový pre efektívnu spoluprácu všetkých komponentov. Počas vývoja bolo otestovaných niekoľko prístupov, kým sa nepodarilo dosiahnuť optimálne riešenie. V tejto kapitole popíšeme použitý dizajn a jeho následnú implementáciu.

6.1 Detekcia vozovky

Detekcia jazdných pruhov a hraníc vozovky predstavuje jeden z kritických subsystémov autonómnych vozidiel. Jej hlavnou úlohou je identifikovať cestu, udržať sa na nej a správne zareagovať v prípade zmien vozovky. AV sú schopné túto funkciu vykonávať fúziou rôznych senzorov [11].

Táto podkapitola sa zaoberá metódami spracovania dát, ktoré umožňujú systému identifikovať vozovku od prostredia. Používame metódy a funkcie dostupné v programovacom jazyku Python, ale aj v knižniciach NumPy a OpenCV. Nainštalovali sme ich pomocou systému na správu balíkov pip príkazmi `pip install opencv-python` a `pip install numpy`.

6.1.1 Region of interest

V kontexte autonómnych vozidiel sa ROI používa na ohraničenie určitej oblasti obrazového vstupu kamier, ktorá je pre daný systém detekcie relevantná [23].

Pri implementácii detekcie vozovky sme použili ROI ako prvý krok pred-spracovania. Vozovka sa v drvivej väčšine prípadov nachádza v spodnej polovici obrazu a spracovanie hornej časti zbytočne vnáša nepotrebné informácie, čo spôsobuje vizuálny šum a zťažuje procesor.

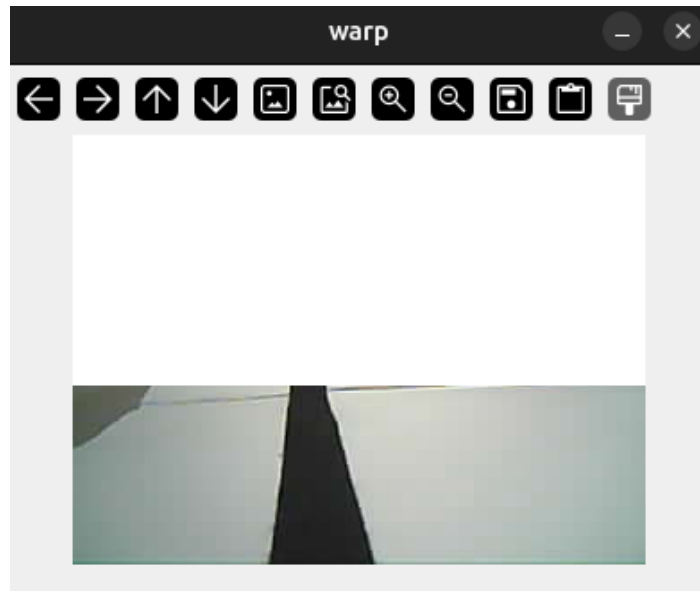
Na oddelenie ROI sme použili funkcie dostupné v knižnici OpenCV.

```
1 def region_of_interest(frame):
2     h, w = frame.shape[:2]
3     roi_height = 100
4     roi_mask = np.ones_like(frame) * 255
5
6     roi_mask[h - roi_height:h, :] = frame[h - roi_height:h, :]
7
8     return roi_mask
```

Výpis kódu 1: Funkcia ROI

Proces začal vytvorením pomocnej matice, s rovnakými rozmermi ako vstupný obraz, pričom všetky jej prvky boli nastavené na maximálnu hodnotu 8-bitového rozsahu (255). Túto maticu sme si definovali pomocou knižnice NumPy. Následne bola do tejto

matice prenesená relevantná časť datasetu od vopred definovanej vertikálnej hranice `roi_height` smerom nadol. Výsledok, viditeľný na obrázku 7, je obraz, kde sú spracované len tie pixely, ktoré sú v spodnej polovici obrazu a pravdepodobne obsahujú informácie o vozovke.



Obr. 7: Obraz s použitím ROI

6.1.2 Hľadanie tmavých regiónov

Hľadanie tmavých regiónov na vozovke je jednoduchý a výpočtovo nenáročný spôsob detekcie jazdných pruhov. Autonómne vozidlá dokážu detegovať svetlý jazdný pruh na tmavom asfalte vďaka kontraste týchto dvoch plôch. V princípe používame rovnaký prístup, no v našej implementácii hľadáme čierny pruh v strede bielej vozovky. Tento postup dosahuje lepšie výsledky v zhoršených svetelných podmienkach [23].

Prvý krok je definícia hraníc pre najsvetlejší a najtmavší tmavý región. V kóde si definujeme hranice ako NumPy pole s reprezentáciou vo formáte odtieň-sýtosť-jas (HSV). Odtieň (hue) predstavuje hodnotu od 0 po 179 – čiže polohu na farebnom kolese. Sýtosť (saturation) je percentuálne vyjadrenie „čistoty“ farby – t.j. množstvo šedej; volíme čísla od 0 po 255. Jas (Value) určuje intenzitu svetla. Hodnota 0 predstavuje absolútnu čiernu, zatiaľ čo hodnota 255 zodpovedá maximálnemu jas farby [23].

```
1 lower_black = np.array([0, 0, 0])
2 upper_black = np.array([180, 255, 70])
```

Výpis kódu 2: Definície maxima a minima tmavých regiónov

V ďalšom kroku konvertujeme obraz z kamery z pôvodného blue-green-red (BGR) formátu na HSV reprezentáciu pomocou OpenCV funkcie `cvtColor` s argumentom

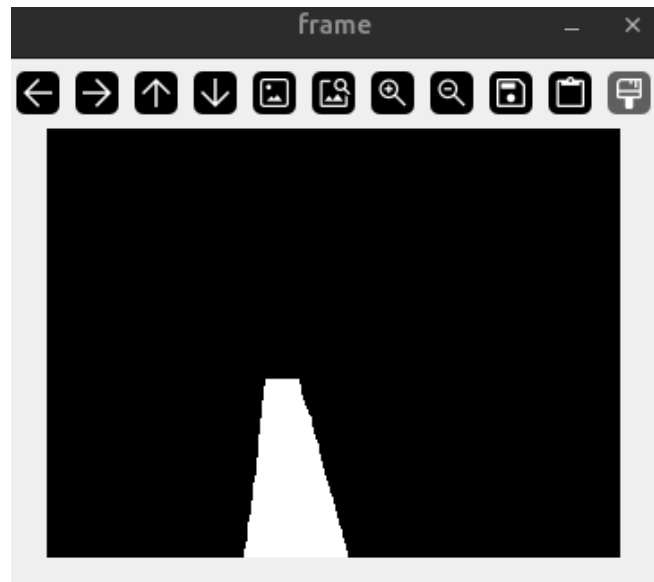
COLOR_BGR2HSV. Volaním funkcie `inRange` s nami definovanými hranicami dostaneme zamaskované tie pixely, ktoré spadajú do tohto rozmedzia [23].

Výstup obsahuje šum a diery, ktoré môžu viesť nesprávne výsledky, preto ich musíme eliminovať. Daný efekt vieme dosiahnuť pomocou morfolologickej transformácie. Šum predstavuje malé izolované regióny pixelov, ktoré do obrazu nepatria. Proces erózie vie tento šum odstrániť – v kóde je to volaním funkcie `morphologyEx` s argumentom `MORPH_OPEN` a maticou jednotiek, ktorá funguje ako filter. Na uzavretie dier – tmavých oblastí vo vnútri objektu – používame rovnakú funkciu, no v tomto prípade s argumentom `MORPH_CLOSE`. Tento postup sa nazýva dilatácia [23].

```
1 def mask_road(frame):
2     hsv = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
3     mask = cv.inRange(hsv, lower_black, upper_black)
4
5     kernel = np.ones((5, 5), np.uint8)
6     mask = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel)
7     mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)
8
9     return mask
```

Výpis kódu 3: Maskovanie tmavých regiónov

Výstupné maskovanie silne závisí od externých faktorov, ako napríklad okolité osvetlenie. Obrázok 8 zobrazuje možný výstup maskovania.



Obr. 8: Hľadanie tmavého jazdného pruhu.

6.1.3 Hľadanie kontúr vozovky

Kontúry sú dôležitou súčasťou detekcie jazdných pruhov na vozovke. Princípom sa podobá hľadaniu hranice tmavého objektu na svetlom pozadí [23].

V knižnici OpenCV existuje funkcia `findContours` na algoritmickú detekciu kontúr v obraze. Funkcia vracia dve polia – výstupné pole kontúr a pole opisujúce hierarchiu topológie obrazu, no túto štruktúru zahadzujeme, pretože nie je pre nás užitočné [23].

Voláme funkciu s dvomi parametrami. Prvý parameter je režim vyhľadávania kontúr; používame mód `RETR_EXTERNAL`, ktorý zachytáva iba extrémne vonkajšie kontúry. Alternatívne režimy zachytávajú všetky možné kontúry, no líšia sa v hierarchickom usporiadaní kontúr – zoznam, dvojstupňová alebo stromová štruktúra [23].

Druhý parameter predstavuje algoritmus aproximácie kontúr. V implementácii je využívaný režim `CHAIN_APPROX_SIMPLE`. Z praktického hľadiska komprimuje segmenty bodov ležiace na jednej rovine – horizontálne, vertikálne alebo diagonálne – a zanechá iba ich koncové body. To znamená, že štvorcová kontúra bude zjednodušená iba na jej rohové body. Iná metóda používa algoritmus aproximácie pomocou Teh-Chin reťazcov, ďalšia ponecháva všetky body kontúry [23]. Implementácia kódu v systéme detekcie čiar vyzerá nasledovne.

```
1 def find_contours(mask):  
2     contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)  
3     return contours
```

Výpis kódu 4: Detekcia kontúr

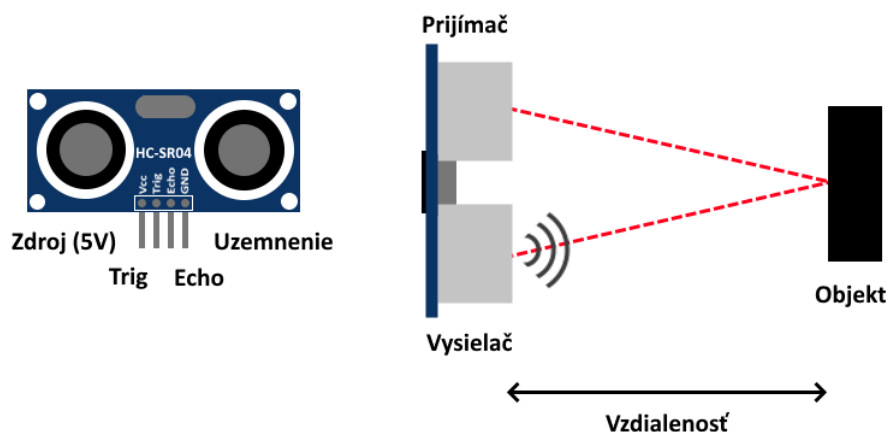
6.2 Detekcia prekážok

Dôležitým pilierom bezpečnosti autonómnych vozidiel je schopnosť detekcie prekážok. V reálnom čase musia identifikovať a klasifikovať prekážku na vozovke alebo v jej blízkosti a adekvátne na ňu reagovať. Tento proces využíva dáta z rôznych senzorov na vyhodnotenie situácie. Na základe týchto dát dokáže AV obísť objekt alebo zastaviť [11].

V tejto podkapitole si vysvetlíme použité metódy na detekciu prekážok.

6.2.1 Ultrazvukový senzor

Pri našej implementácii sme použili ultrazvukový senzor na detekciu prekážok priamo pred vozidlom. Použitý HC-SR04 ultrazvukový senzor funguje ako vysielač, prijímač aj riadiaci obvod – obrázok 9 ilustruje jeho funkciu.



Obr. 9: Diagram fungovania ultrazvukového senzora

Princíp merania vzdialenosti je nasledovný:

1. Arduino pošle signál vysokej úrovne na **Trig** pin ultrazvukového senzora.
2. Modul pošle osem 40kHz signálov a nastaví vysokú úroveň na pine **Echo**.
3. Spustí sa časovač, zatiaľ čo senzor očakáva návratový signál.
4. Ak zaznamená pulz, zastaví časovač a zníži hodnotu **Echo** pinu.

Keďže máme rýchlosť zvuku ($343m \cdot s^{-1}$) a čas, za ktorý sa signál vrátil, vieme vypočítať vzdialenosť vozidla a objektu pred ním. Rovnica na výpočet vzdialenosti je nasledovná:

$$\text{vzdialenosť} = \frac{\text{nameraný čas}}{2} \cdot \text{rýchlosť zvuku}$$

Ultrazvukový modul riadime pomocou dvoch pinov mikrokontroléra Arduino. Jeden pin, nastavený na mód OUTPUT, používame sa zaslanie vysokej hodnoty na modul. Druhý pin, nastavený na mód INPUT, je určený na získanie signálu zo senzora; na tento pin sme pripojili rutinu prerušenia.

```

1 void IRAM_ATTR echoISR() {
2   if (digitalRead(ECHO_PIN)) {
3     echoStart = micros();
4   } else {
5     echoEnd = micros();
6     echoDone = true;
7   }
8 }

```

Výpis kódu 5: Obsluha prerušenia ultrazvukového senzora

Keďže je tento výpočet náročnejší, merania prebiehajú v 50 ms intervaloch. Samotný výpočet sa vykonáva v rámci danej iterácie hlavného cyklu.

```
1  if (echoDone) {
2      uint32_t duration = echoEnd - echoStart;
3      distance = duration / 58;
4      echoDone = false;
5  }
```

Výpis kódu 6: Výpočet vzdialenosti z merania ultrazvukového senzora

Z dôvodu zjednodušenia výpočtu používame aproximáciu skutočnej vzdialenosti v centimetroch. Ak sa meraný predmet nachádza vo vzdialenosti do 10 cm, vozidlo zastaví [18].

6.2.2 Ultralytics YOLOv8

V našej implementácii používame YOLOv8 od spoločnosti Ultralytics – ako bolo popísané v sekcii 5.5.1. Pre programovací jazyk Python existuje jednoduché rozhranie, ktoré prácu s modelom zjednodušuje. Inštalácia prebehla nasledovným príkazom: `pip install ultralytics`

Nasledovný kód nám vytvorí inštanciu predtrénovaného modelu `yolov8s.pt` a zadefinuje funkciu, pomocou ktorej vieme detegovať objekty.

```
1 model = YOLO("yolov8s.pt")
2
3 def detect(frame):
4     return model(frame, verbose=False)[0]
```

Výpis kódu 7: Príprava CNN na detekciu objektov

Funkcia `detect` vracia zoznam (list) objektov `Result`, napriek tomu, že vstupom je iba jedna snímka. Vyberieme nultý prvok, ktorý obsahuje všetky detekcie pre daný obrázok.

6.3 Interpretácia dát

Správna interpretácia výstupných dát z modulov je dôležitá pre spolunažívanie senzorov a pohybových systémov vozidla. V autonómnych vozidlách ide o modul plánovania a rozhodovania, ktorý má za úlohu prepojiť podsystemy vnímania a konania.

V prípade, že sa kamera pripojila a dokážeme načítať jej výstup, môžeme pokračovať s jeho spracovaním.

Začneme s detekciou prekážok pomocou CNN. Z dôvodu zvýšenia výpočtovej efektivity programu spúšťame detekciu každých 6 snímok. Model YOLOv8 je schopný

detegovať 80 rôznych tried objektov, vrátane tých, ktoré by sme na vozovke neočakávali. Aby sme nedostávali falošné pozitíva, vybrali sme si podmnožinu týchto tried na detekciu. Do výberu sme zahrnuli ľudí, dopravné prostriedky a zvieratá.

Za okolností, že model YOLO identifikoval objekt z definovaného zoznamu prekážok, systém následne vyhodnocuje podmienky pre zastavenie vozidla. Rozhodovací systém je založený na troch kľúčových podmienkach, ktoré boli založené na základe testovania a optimalizácie systému:

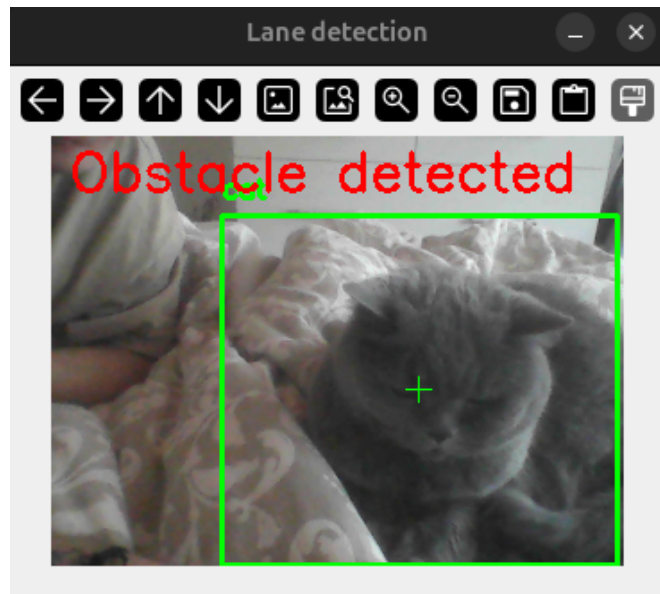
1. **Trajektória** Prvým krokom je určenie pozície prekážky vzhľadom na jazdnú dráhu vozidla. Podmienka je splnená v prípade, že sa objekt nachádza v rámci definovanej hranice. Hranice sa zväčšujú podľa blízkosti objektu k vozidlu.
2. **Blízkosť objektu** V druhom kroku zisťujeme plochu, ktorú prekážka zaberá na obrazovke. Ak táto hodnota prekročí stanovenú hranicu, vyhodnotí sa podmienka za splnenú. Pre každú z vybraných tried je definovaná vlastná hranica.
3. **Spôľahlivosť** Záverečná podmienka je overenie miery konfidencie modelu. Týmto sa minimalizujú falošné pozitíva a nežiadúce zastavenie vozidla.

Až po súčasnom splnení všetkých troch podmienok sa aktivuje zastavenie vozidla.

```
1 def decide_stop(det):
2     for box in det.bboxes:
3         cls = int(box.cls[0])
4         name = model.names[cls]
5
6         if name in obstructions:
7             x1, y1, x2, y2 = map(int, box.xyxy[0])
8
9             area = (x2 - x1) * (y2 - y1)
10            conf = float(box.conf[0])
11            margin = int(area * 0.001)
12
13            in_front = x1 < (LANE_RIGHT + margin) and x2 > (LANE_LEFT - margin)
14            close = area > obstructions[name]
15            reliable = conf > CONF_THRESHOLD
16
17            if in_front and close and reliable:
18                return True, box
19    return False, None
```

Výpis kódu 8: Rozhodovanie zastavenia pri detekcii pomocou CNN

Posledný krok je zaznačiť detekciu na snímke – viď obrázok 10.



Obr. 10: Detekcia prekážky pomocou CNN.

V alternatívnom scenári, kde nebola nedetegovaná prekážka pomocou CNN, budeme postupovať s určením jazdného pruhu.

Po identifikácii kontúr jazdného pruhu je ďalším krokom určenie stredovej línie tej najvýraznejšej z nich. Vyberieme si kontúru s najväčšou plochou. Pomocou funkcie `moments` z knižnice OpenCV získame momenty pixelov – vážené priemery intezity (jasu) jednotlivých pixelov. Tieto údaje sú kľúčové pri výpočte geometrického stredu kontúry [23].

Súradnice stredu (c_x, c_y) vypočítame pomocou nasledujúcich vzorcov:

- **X-ová súradnica** $c_x = \frac{m_{10}}{m_{00}}$
- **Y-ová súradnica** $c_y = \frac{m_{01}}{m_{00}}$

Kde m_{10} a m_{01} predstavujú horizontálny a vertikálny moment a m_{00} zodpovedá ploche danej kontúry.

```

1 def find_center(contours):
2     if contours:
3         largest = max(contours, key=cv.contourArea)
4         M = cv.moments(largest)
5         if M["m00"] != 0:
6             cx = int(M["m10"] / M["m00"])
7             cy = int(M["m01"] / M["m00"])
8
9         return cx, cy
10    return None, None

```

Výpis kódu 9: Výpočet geometrického stredu kontúry

Po zistení geometrického stredu jazdného pruhu vypočítame samotnú hodnotu riadenia. Naša implementácia normalizuje hodnoty na rozsah hodnôt $< 1,511 >$, čím zabezpečíme plynulé vstupy pre riadiaci systém vozidla.

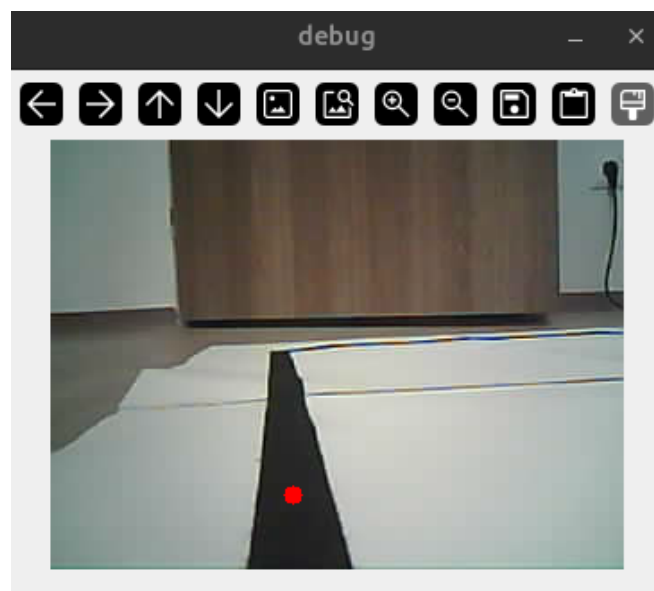
- **Hodnoty $< 1,255 >$:** Otáčanie doľava.
- **Hodnoty $< 257,511 >$:** Otáčanie doprava.
- **Hodnota 256:** Priama jazda.
- **Hodnota 0:** Rezervovaná pre zastavenie.

Tento normalizovaný postup priraduje odchýlku zo stredu priamo k hodnote riadenia. Aplikáciou nelineárnej mocninateľnej funkcie transformujeme výstupný rozsah tak, aby sme v okolí stredovej osi dosiahli vyššiu stabilitu a presnosť riadenia.

```
1 def compute_steering(cx, center=160, alpha=1.6):
2     error = (cx - center) / center
3     error = pow(abs(error), alpha) * (1 if error >= 0 else -1)
4
5     return int(256 + error * 255)
```

Výpis kódu 10: Výpočet hodnoty riadenia

Pomocou zistených hodnôt vieme zaznačiť stred jazdného pruhu a vypísať hodnotu otáčania, ako je možné vidieť na obrázku 11.



Obr. 11: Hľadanie stredu jazdného pruhu.

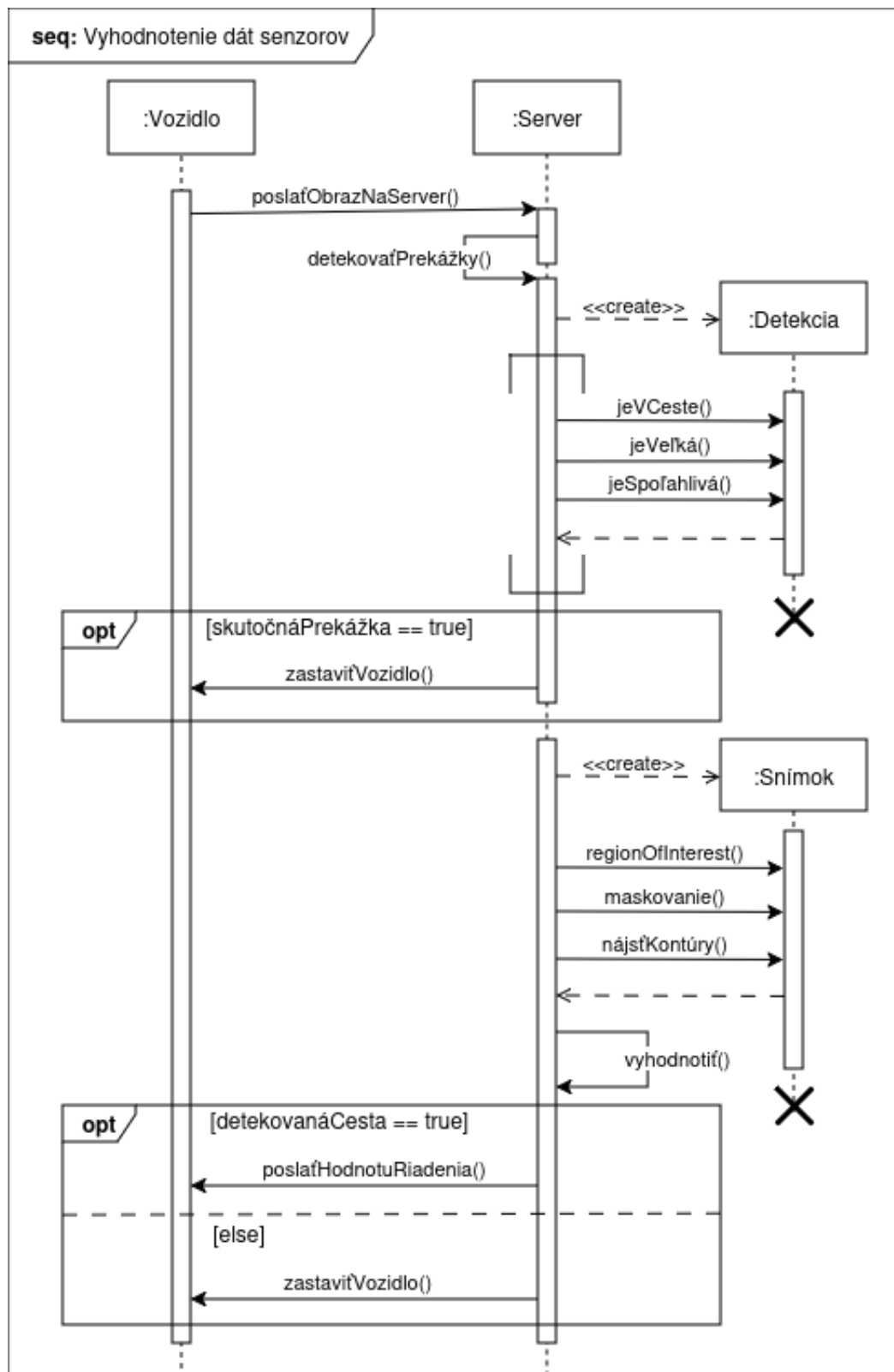
Následne sa táto hodnota posiela na Arduino mikrokontrolér.

Na Arduine sa najskôr skontroluje hodnota z ultrazvukového modulu. V prípade, že je detegovaná prekážka do desať centimetrov, vozidlo zastaví. Ultrazvukový senzor berie najvyššiu prioritu.

```
1 if (distance < 10 || steer < -255 || steer > 255) {  
2   stopWheels();  
3 } else {  
4   driveSteering(steer);  
5 }
```

Výpis kódu 11: Rozhodovanie na vozidle

Celkovú postupnosť krokov v tejto podkapitole opisuje nasledovný sekvenčný diagram.

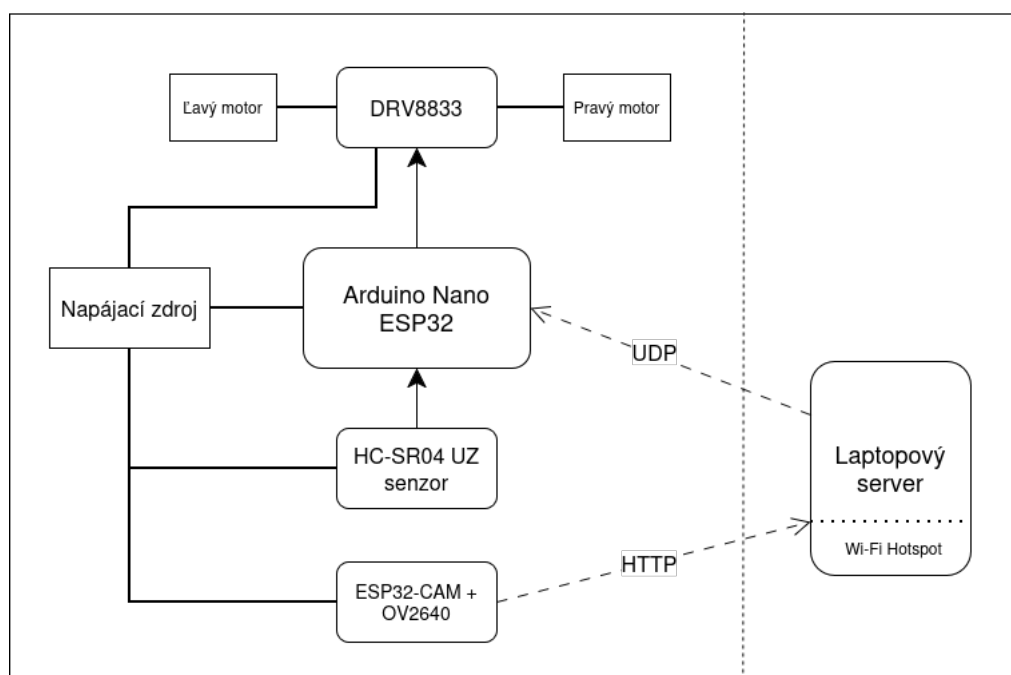


Obr. 12: Sekvenčný diagram vyhodnotenia dát zo senzorov.

6.4 Návrh komunikácie komponentov

Komunikácia komponentov predstavuje dôležitý aspekt projektu, pretože je nutné aby si jednotlivé časti mohli synchronne vymieňať a spracovať dáta. Systém je navrhnutý tak, aby v prípade výpadku jedného alebo viacerých komponentov dokázal bezpečne zastaviť vozidlo.

Dôsledkom nízkeho výpočtového výkonu mikrokontrolérov Arduino sme použili v našej implementácii model klient-server. V danej štruktúre vystupujú dva typy entít; nadradení klienti iniciujú požiadavky na pasívny server, ktorý posiela odpovede klientom.



Obr. 13: Diagram komunikácie komponentov

6.4.1 Komunikácia kamery a serveru

Server je v našom projekte laptop, ktorý taktiež spúšťa Wi-Fi sieť na komunikáciu s komponentmi – viď obrázok 13. Všetky funkcie webserveru sme programovali pomocou jazyka Python. Bezdrôtový prípojný bod sme vytvorili NetworkManagerom, nástrojom na konfiguráciu sieťových zariadení v operačnom systéme Linux, pomocou ktorého sme sieťovú kartu nášho laptopu zmenili na prípojný bod Wi-Fi hotspotu. Na spustenie tohto programu v Pythone sme použili knižnicu subprocess.

```

1 SSID = "LAPTOP_AP"
2 PASSWORD = "12345678"
3
4 def startHotspot():
5     print("Starting Hotspot")
6     subprocess.run([
7         "nmcli",
8         "device",
9         "wifi",
10        "hotspot",
11        "ssid", SSID,
12        "password", PASSWORD
13    ], check=True)

```

Výpis kódu 12: Zapnutie Wi-Fi hotspotu

Pred spustením cyklu spracovania dát musíme počkať kým sa kamerový modul pripojí na Wi-Fi sieť. To dokážeme zistiť pomocou Internet Control Message Protokolu (ICMP). Vysielame ping pakety v rozmedzí 1 sekundy. Výstup 0 znamená úspech a vieme, že sa kamera úspešne pripojila.

```

1 def ping_cam():
2     while True:
3         response = os.system(f"ping -c 1 {CAM_IP} > /dev/null 2>&1")
4         if response == 0:
5             print("ESP32-CAM online")
6             return
7         print(".", end="")
8         time.sleep(1)

```

Výpis kódu 13: Pingovanie IP adresy kamerového modulu

ESP32-CAM pracuje ako klient, ktorý posiela obrázkové dáta na server za pomoci Wi-Fi. Po pripojení na server začne cez protokol HTTP na porte 81 vysielat video z kamery OV2640. Obrázky sú rozlíšenia QVGA (t.j 320x240 pixelov) s formátom JPEG. Nový obrázok sa pošle každých 30ms, čo znamená, že stream má približne 30FPS.

```

1 void startCameraServer(){
2     httpd_config_t config = HTTPD_DEFAULT_CONFIG();
3     config.server_port = 81; // port streamu
4
5     if(httpd_start(&stream_httpd, &config) == ESP_OK){
6         httpd_uri_t stream_uri = {
7             .uri          = "/stream",
8             .method        = HTTP_GET,
9             .handler        = stream_handler,
10            .user_ctx       = NULL
11        };
12        httpd_register_uri_handler(stream_httpd, &stream_uri);
13    }
14 }

```

Výpis kódu 14: Spustenie HTTP stream na ESP32-CAM

6.4.2 Komunikácia serveru a mikrokontroléra

Vďaka statickej IP adrese dokážeme serveru povedať na akej URL má očakávať obrázky. Funkcia VideoCapture z knižnice OpenCV má podporu pre spracovanie obrazu z URL. Po interpretovaní dát, server pošle požiadavku na Arduino Nano. Odpoveď sa odošle cez UDP socket, ideálne pre aplikácie fungujúce v reálnom čase, IP adrese mikrokontroléra na porte 5005. Komunikáciu sprostredkuje Python knižnica socket. Poslané dáta sú vo forme 2 bytov, čo je najmenší potrebný buffer na uloženie údajov o riadení.

```

1 CAM_IP = "10.42.0.128"
2 CAM_URL = f"http://{CAM_IP}:81/stream"
3
4 UDP_IP = "10.42.0.111"
5 UDP_PORT = 5005
6
7 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
8
9 def sendUDP(direction: float):
10     sock.sendto(struct.pack('<H', direction), (UDP_IP, UDP_PORT))

```

Výpis kódu 15: Posielanie UDP packetov

6.4.3 Komunikácia mikrokontroléra s perifériami

Arduino prijme UDP packet a súčasne získa nameranú vzdialenosť z ultrazvukového senzora. Keďže Echo pin na senzore, z ktorého získavame signál, má výstupné napätie 5V, musíme ho znížiť na 3.3V, čo je operačné napätie všetkých GPIO pinov mikrokontroléra.

Toto sme dosiahli pomocou deliča napätia. Výstupné napätia deliča vypočítame pomocou rovnice $V_{OUT} = \frac{V_{IN} \cdot R_2}{R_1 + R_2}$, kde R_1 a R_2 sú odpory dvoch rezistorov zapojených sériovo, V_{IN} je vstupné napätie a V_{OUT} výstupné napätie. Určili sme, že potrebujeme zapojiť $10\text{ k}\Omega$ a $20\text{ k}\Omega$ odpor v sérii.

Ultrazvukový snímač sa spúšťa v intervaloch 50ms, z dôvodu efektívneho využitia výpočtového výkonu. Jednotlivé spustenia sú vo forme vyvolania prerušenia.

Každému z motorčekov sú priradené dva výstupné piny – pinA a pinB. Všetky štyri piny sú priradené na osobitný PWM kanál. Pulse Width Modulation (PWM) nám dovoľuje presnú kontrolu nad rýchlosťou motorčekov. Kanály sme nakonfigurovali pomocou funkcie `ledcSetup`. V danej funkcii sme nastavili frekvenciu menenia (PWM_FREQ) na 20kHz a rozlíšenie (PWM_RES) na 8. Takéto rozlíšenie nám dá 256 (2^8) rôznych stupňov rýchlosti. Následne sme funkciou `ledcAttachPin` priradili piny na kanály.

Signály z pinov sú zaslané na DRV8833, ktorý fyzicky poháňa dva DC motory. Na tento účel používame funkciu `ledcWrite` z knižnice funkcií Arduino pre ESP32 mikrokontroléry. V prípade, že sa snažíme poháňať motor dopredu napíšeme na pinA hodnotu rýchlosti a pinB nastavíme na hodnotu 0. V opačnom scenári sa zápisy pre piny vymenia. Ak sa snažíme motorček zastaviť, zapíšeme na obidve piny 0, no zápisy vysokých hodnôt by mali rovnaký efekt.

```

1 void loop() {
2   while (udp.parsePacket() >= 2) {
3     udp.read(incoming, 2);
4
5     uint16_t raw = incoming[0] | (incoming[1] << 8);
6     steer = (int)raw - 256;
7   }
8
9   static unsigned long lastTrig = 0;
10  if (millis() - lastTrig > 50) {
11    lastTrig = millis();
12    echoDone = false;
13    triggerUltrasonic();
14  }
15
16  if (echoDone) {
17    uint32_t duration = echoEnd - echoStart;
18    distance = duration / 58;
19    echoDone = false;
20  }
21 }

```

Výpis kódu 16: Vnútorý cyklus Arduina

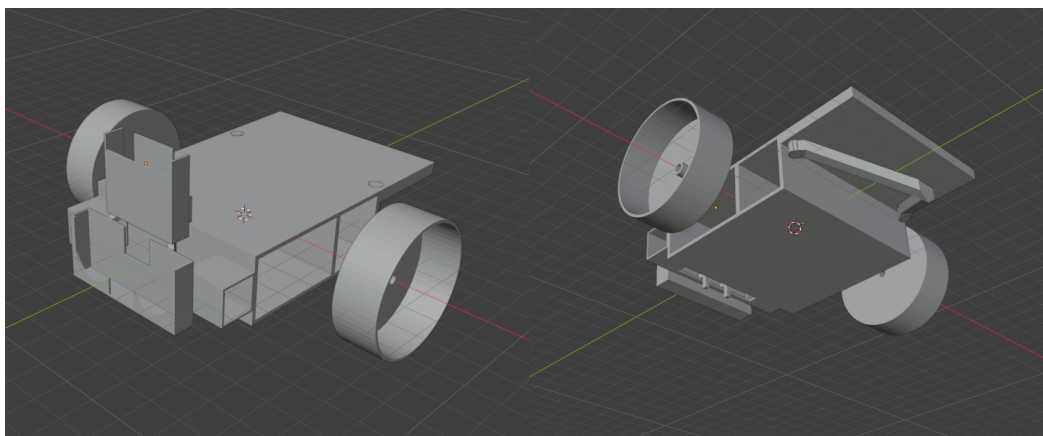
6.5 Návrh rámu a rozloženie komponentov

Správny návrh rámu a rozloženia komponentov na ňom je dôležitý pre správne fungovanie celého systému. V našej implementácii sme sa zamerali na minimalizovanie hmotnosti vozidla a maximalizovanie kompaktnosti komponentov, a tým ich zohranie.

Na návrh rámu sme použili softvér Blender – populárny, všeobecne používaný program na dizajn, modelovanie a animáciu, ktorý umožňuje exportovať do formátu .stl, ideálneho pre 3D tlač. Po odmeraní všetkých komponentov, sme začali pracovať na prototype rámu.

Vyskúšali sme mnoho návrhov, kde sme otestovali rôzne rozloženia a orientácie súčiastok. Zvolili sme si dizajn s dvoma kolesami vpredu a zadnou vidlicou, ktorá je vymeniteľná (pre prípad opotrebovania alebo zmeny povrchu).

Najťažšie komponenty – batériu a menič napätia – sme umiestnili na spodok vozidla, aby sme znížili ťažisko a zvýšili stabilitu. Pridali sme držiak na kameru a ultrazvukový senzor na stabilizáciu obrazu.



Obr. 14: 3D model vozidla

Model bol vytlačený na tlačiarni Bambulab A1 mini, použitím filamentu kyseliny polymliečnej (PLA). Prvotne boli kolesá vytlačené z polyetyléntereftelátu (PET) a doplnené o protišmykový polep, no tento povrch nedosahoval požadovaný stupeň trakcie.

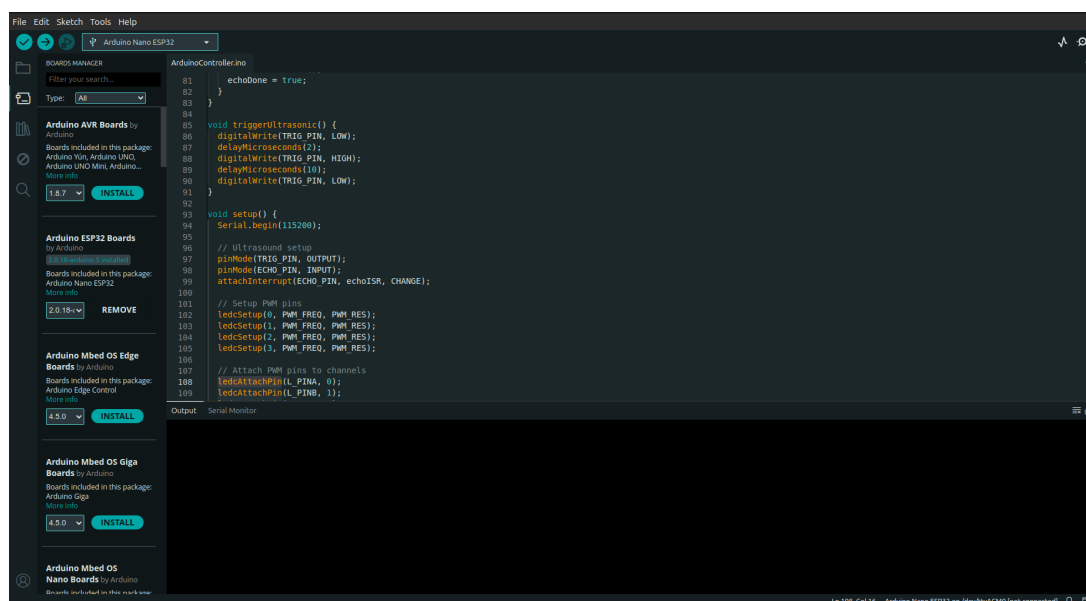
Zakúpili sme dve kolesá s priemerom 43mm, gumeným povrchom a oskou určenou pre náš typ motorčekov.

6.6 Montovanie vozidla a nahratie softvéru

V tejto podkapitole si priblížime fyzické spojenie hardvéru a softvéru. Priebeh procesu nebol priamočiary a vykazoval časové aj logické odchýlky od predpokladaného modelu. Mnohé spomenuté komponenty boli testované samostatne a skutočný program prešiel

mnohými iteráciami, predtým než sme našli tú ideálnu. Pre potreby tejto práce však uvádzame optimálnu sekvenciu krokov, ktoré predstavujú logicky správny postup.

Začali sme nahratím kódu na Arduino Nano ESP32. Tento mikrokontrolér má podporu nahrávania kódu pomocou konektoru USB-C. Pomocou dátového USB káblu sme zapojili Arduino do laptopu, kde máme uložený program. Integrované vývojové prostredie Arduino IDE (obrázok 15) automaticky detegovalo pripojený mikrokontrolér. Pomocou tlačidla Upload v tomto prostredí sme úspešne skompilovali a nahrali kód na Arduino Nano ESP32.



Obr. 15: Integrované vývojové prostredie Arduino

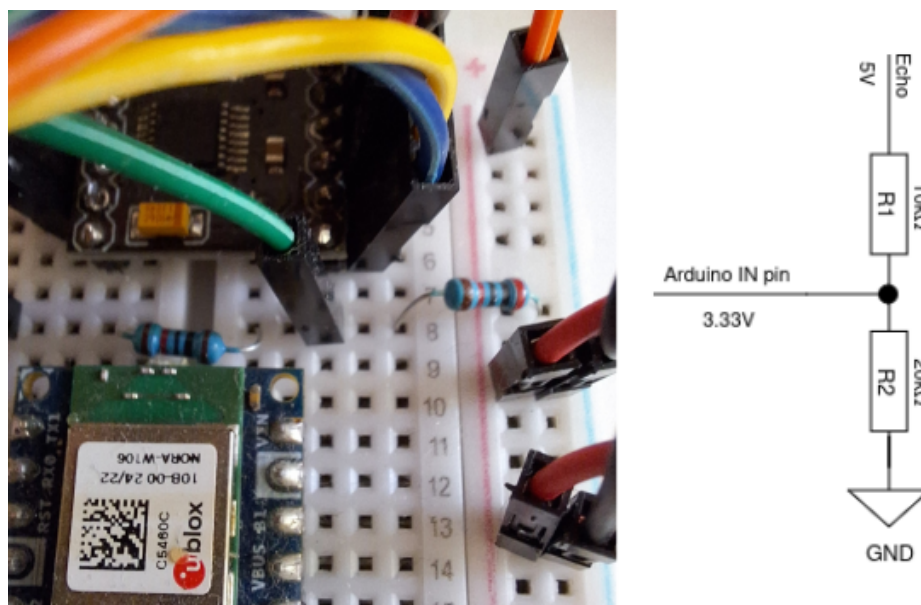
V ďalšom kroku sme nahrali kód na ESP32-CAM. Narozdiel od Arduina, ESP32-CAM nemá univerzálnu zbernicu. Na tento účel sme pripojili programovací shield, určený pre túto dosku. Tento modul funguje ako adaptér USB na sériové rozhranie a disponuje starším Micro USB B portom. Podobne ako v minulom kroku, Arduino IDE rozpoznalo vývojovú dosku. Kód sme skompilovali a nahrali pomocou tohto prostredia. Týmto krokom je nahratie softvéru na časť vozidla hotové.

Na prepojenie súčiastok sme použili nepájivé pole a farebne odlíšené vodiče. Pripevnili sme ho na rám vozidla pomocou lepiacej vrstvy na spodnej strane.

Vďaka headerom sme zapojili Arduino Nano ESP32 a motor driver DRV8833 na breadboard.

Podľa definície sme spojili výstupné štyri piny Arduina pre ľavý a pravý motorček k IN pinom motor drivera. K motorčekom sme prispájkovali káble a pripevnili kolesá. Celú sústavu pre obe strany sme následne zasunuli do vyhradených držiakov. Koncové hroty sme zapojili k OUT pinom motor drivera.

V ďalšom kroku sme zapojili ultrazvukový senzor. Trig pin senzora sme spojili s dezinovaným output pinom na Arduino pomocou vodiča. Zapojili sme $10\text{ k}\Omega$ rezistor na Echo pin senzora a $20\text{ k}\Omega$ cez zdieľané uzemnenie. Výstupné napätie 3.333V – prijateľné pre Arduino – sme získali z vodiča pripojeného medzi dvomi odpormi. Koniec vodiča sme zapojili do input pinu na Arduino – viď obrázok 16.



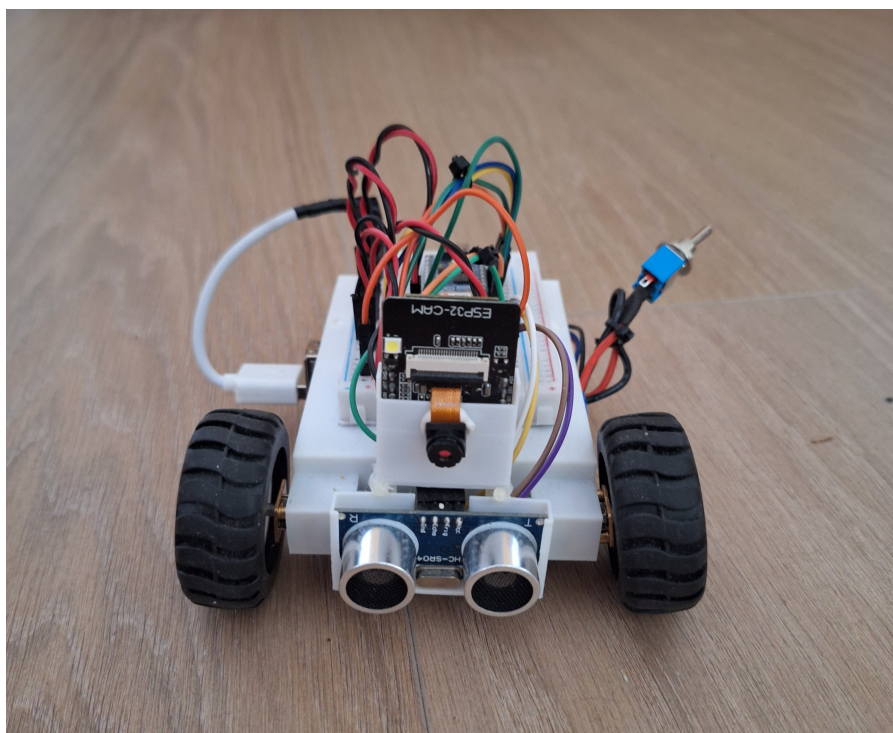
Obr. 16: Delič napätia

Ultrazvukový senzor a kamerový modul sme vložili do držiaka. Následne sme držiak nasadili na dva podporné body nachádzajúce sa na ráme vozidla. Pomocou sťahovacích pásov sme držiak pripevnili.

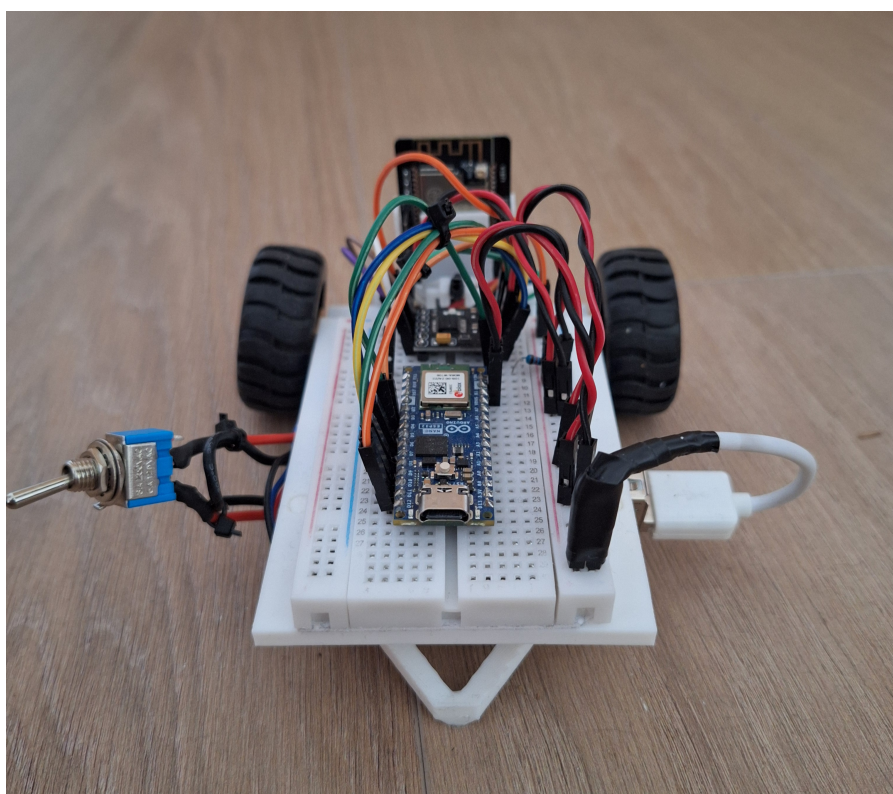
V ďalšom kroku sme zapojili batériu. Na vodič kladného pólu batérie sme pripevnili prepínač. Následne obidva káble sme zapojili do buck meniča. K výstupnému portu sme pripojili modifikovaný USB kábel. Koncový konektor sme odstránili; k červenému a čiernemu vodiču sme pripevnili sondy – kladná a záporná. Nový konektor sme zapojili do napájacej koľajnice.

Mikrokontrolér, kamerový modul, motor driver a ultrazvukový senzor boli zapojené na napájaciu koľajnicu pomocou čiernych a červených káblikov.

Na záver sme do rámu vložili podpornú vidlicu, čím sme montáž vozidla skompletovali – viď obrázok 17 a 18.



Obr. 17: Skompletizované vozidlo (predok)



Obr. 18: Skompletizované vozidlo (zadok)

7 Metodika testovania

V tejto kapitole sú popísané použité techniky a proces testovania navrhnutého systému. Testovanie je nevyhnutnou súčasťou realizácie každého projektu, pretože umožňuje overiť funkčnosť návrhu v praxi a posúdiť mieru naplnenia stanovených cieľov v reálnych podmienkach. Výsledky tejto časti zároveň slúžia na porovnanie dosiahnutých riešení s inými prácami, ktoré využívajú odlišnú architektúru.

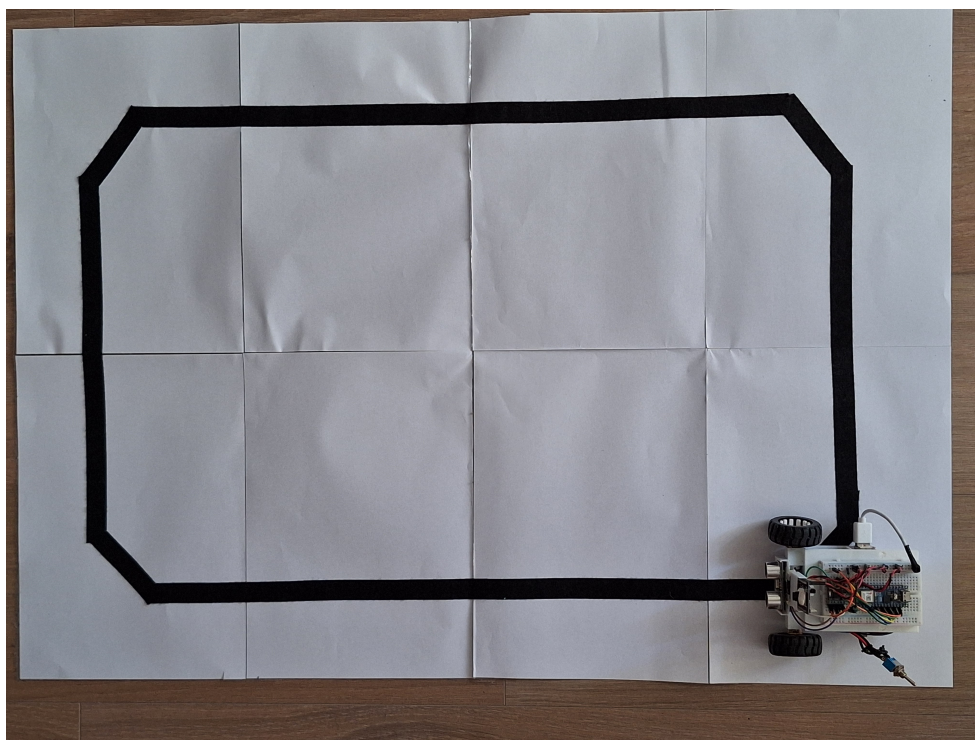
V rámci experimentov sme sa zamerali na overenie dvoch hlavných funkcií systému:

- **Autonómne riadenie** Cieľom bolo otestovať schopnosť vozidla sledovať vopred určenú trasu. Testovacia dráha bola navrhnutá tak, aby obsahovala rovné úseky aj zákruty. Úspešnosť systému sme hodnotili podľa stability a schopnosti vozidla udržať sa na vozovke bez vybočenia.
- **Detekcia prekážky** Keďže vývoj tohto systému bol hlavným cieľom práce, väčšina testov je zameraná práve na túto oblasť. Systém využíva dva typy senzorov – kamerový modul s konvolučnou neurónovou sieťou a ultrazvukový senzor. Testy sú navrhnuté tak, aby preverili spoľahlivosť oboch týchto systémov.

7.1 Autonómne riadenie

Autonómne riadenie chápeme, ako schopnosť vozidla samostatne vykonávať jazdné úkony bez zásahu ľudského operátora. Pre overenie tejto schopnosti musí riadiaci systém vozidla zvládať jazdu nielen na priamych úsekoch, ale aj v zákrutách. Z tohto dôvodu bolo nevyhnutné navrhnuť testovaciu dráhu, ktorá zahŕňa obidva prvky. Grafický návrh trate bol pôvodne vypracovaný v prostredí editora obrázkov.

Po grafickom návrhu nasledovala fyzická realizácia trate. Na jej vytvorenie sme použili osem papierov formátu A4 (celkovou plochou zodpovedajúcich formátu A1), ktoré boli vzájomne prepojené do súvislého celku. Samotná dráha bola následne vytvorená pomocou čiernej matnej lepiacej pásky, pričom sme ako presnú predlohu využili digitálny dizajn (obrázok 19).



Obr. 19: Testovacia trať

Proces testovania spočíva v sledovaní dvoch hlavných parametrov: schopnosti vozidla prejsť celú dráhu bez jej opustenia a času, za ktorý okruh absolvuje. Meranie prebiehalo v interiérovyh podmienkach pri ambientnom dennom svetle. Čas prejazdu vozidla cez stanovenú dráhu sme merali od momentu prvotného pohybu až po opätovné prekročenie štartovacej čiary (dokončenie okruhu).

Pri testovaní sme sledovali aj chybovosť, konkrétne nedokončenie dráhy. Tento jav sme definovali v dvoch prípadoch:

- ak sa vozidlo permanentne zastavilo v dôsledku straty detekcie vodiacej čiary,
- ak vozidlo opustilo jazdný pruh z dôvodu chybné detekcie objektov mimo dráhy.

V rámci tohto testu detekciu prekážok sme nesledovali. Prípadné zastavenie sa preto nepovažovalo za chybu a test bol zopakovaný.

Test sme vykonali celkovo desaťkrát. Všetky namerané výsledky zaznamenali do tabuľky, aby sme mohli následne vyhodnotiť úspešnosť a stabilitu riadenia.

7.2 Detekcia prekážky

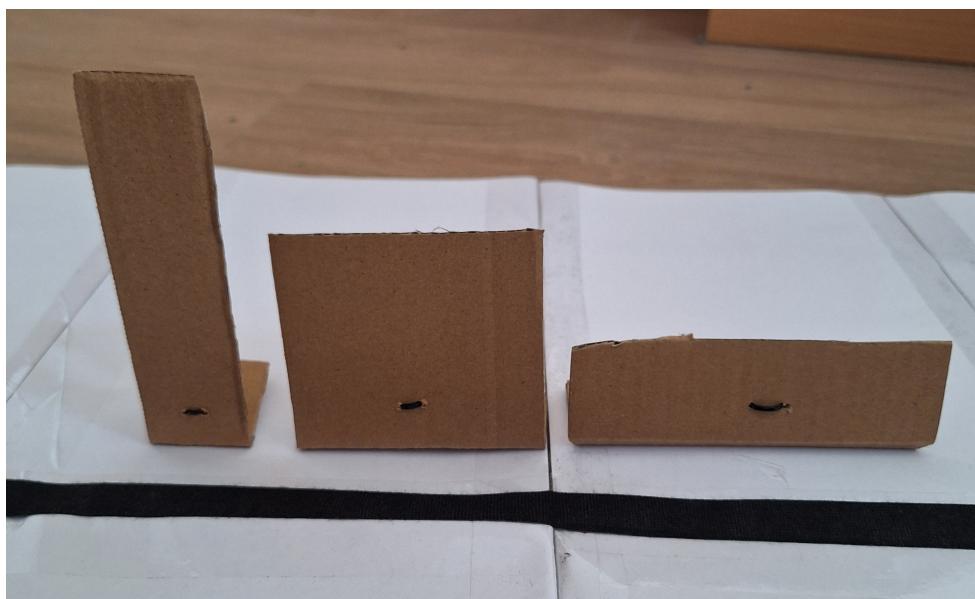
Významnou vlastnosťou autonómneho riadenia je schopnosť vozidla detegovať prekážky a vhodne na ne reagovať. Väčšina systémov využíva určitú mieru redundancie, čo

znamená, že vozidlo disponuje záložným riešením pre prípad zlyhania niektorého z podsystémov.

Z tohto dôvodu sme testovanie rozdelili do dvoch častí:

- Testovanie ultrazvukového modulu.
- Testovanie konvolučnej neurónovej siete.

Na obrázku 20 sú znázornené objekty využité pri testovaní ultrazvukového senzoru.



Obr. 20: Prekážky na testovanie ultrazvukového modulu.

Konkrétne ide o tieto typy prekážok:

- **Štíhla prekážka** (4x14 cm): Slúži na simuláciu objektov s vertikálnou orientáciou, ako sú napríklad stĺpy alebo postavy osôb.
- **Stredne veľká prekážka** (9x9 cm): Predstavuje univerzálny typ objektu a simuluje rozmerovo priemerné prvky prostredia, napríklad časti stien či vozidiel.
- **Široká prekážka** (14x4 cm): Je určená na simuláciu nízkych prekážok s horizontálnou orientáciou, kam môžeme zaradiť nízke oplotenie, menšie zvieratá alebo deti.

Proces testovania bol rozdelený do troch fáz podľa umiestnenia prekážky vzhľadom na jazdnú dráhu:

- **Fáza 1:** Prekážka je umiestnená priamo v strede vozovky.

- **Fáza 2:** Prekážka sa nachádza na okraji vozovky.
- **Fáza 3:** Prekážka je situovaná mimo vozovky.

V každej z týchto fáz sme sledovali a vyhodnocovali reakciu vozidla, konkrétne jeho schopnosť včas zastaviť, respektíve pokračovať v jazde v závislosti od polohy detegovaného objektu. Každý test sme opakovali päťkrát, čím sme získali celkovo štyridsaťpäť meraní.

Testovanie neurónovej siete prebiehalo s piatimi rôznymi objektmi: osobným automobilom, nákladným vozidlom, bicyklom, človekom a psom (obrázok 21). Na testovanie sme použili CNN YOLOv8 od spoločnosti Ultralytics. Sledovali sme nielen úspešnosť detekcie, ale aj to, či systém správne vyhodnotí polohu prekážky (priamo na vozovke alebo v jej blízkosti) a adekvátne na ňu zareaguje. Ultrazvukový senzor sme odpojili, aby jeho vstup neovplyvňoval výsledky neurónovej siete. Každú situáciu sme testovali päťkrát, čo pri kombinácii typu objektu a jeho polohy prinieslo spolu päťdesiat výsledkov.



Obr. 21: Prekážky na testovanie CNN.

Celkovo v tejto časti vyhodnocujeme deväťdesiatpäť testovacích scenárov. Na ich základe určíme efektivitu navrhnutej implementácie a identifikujeme prípadné nedostatky systému.

8 Výsledky

V tejto kapitole sa zameriavame na analýzu dát získaných počas meraní opísaných v predchádzajúcej časti práce. Cieľom objektívneho zhodnotenia je získať relevantnú spätnú väzbu o úspešnosti navrhnutej implementácie. Namerané údaje zároveň slúžia ako podklad pre prípadné porovnanie s inými modelmi autonómnych vozidiel.

Proces vyhodnocovania sme rozdelili do troch častí, ktoré predstavujú jednotlivé systémy riadenia:

- Testovanie detekcie vozovky.
- Testovanie ultrazvukového senzora.
- Testovanie konvolučnej neurónovej siete.

8.1 Testovanie detekcie vozovky

Tabuľka 1: Hodnoty: Detekcia jazdného pruhu

Poradie	Čas	Nedokončilo?
1	11.88 s	nie
2	12.35 s	nie
3	n/a	áno
4	13.13 s	nie
5	13.80 s	nie
6	12.48 s	nie
7	14.84 s	nie
8	12.89 s	nie
9	11.92 s	nie
10	11.38 s	nie
Priemer	12.74 s	90%

Tabuľka 1 zobrazuje výsledky testovania detekcie jazdného pruhu. Z desiatich testov bolo deväť úspešných, kde vozidlo neopustilo trať. V treťom teste sme zaznamenali zlyhanie, kedy vozidlo pri zatáčaní stratilo vizuálny kontakt s jazdným pruhom. Vozidlo nedokázalo znovu detegovať vozovku, a preto sme test označili ako neúspešný.

Priemerný čas dosiahol 12.74 s, pričom medián bol 12.48 sekundy. Rozmedzie výsledkov bolo 3.46 sekúnd. Najrýchlejší prejazd sme zaznamenali v desiatom pokuse (11.38 s). Najpomalší čas bol v siedmom pokuse (14.84 s). Tento údaj je významný,

keďže vozidlo v dôsledku dočasnej straty detekcie pruhu zastalo, čo predĺžilo prejazd o viac než dve sekundy oproti priemeru.

8.2 Testovanie ultrazvukového senzora

Tabuľka 2: Hodnoty : Ultrazvukový senzor – stred vozovky

	Zastavilo?		
Poradie	Štíhla	Stredná	Široká
1	áno	áno	áno
2	áno	áno	áno
3	áno	áno	áno
4	áno	áno	áno
5	áno	áno	áno
Úspešnosť	100.00%	100.00%	100.00%

Tabuľka 2 hodnoty namerané pri umiestnení prekážky na stred vozovky. Ultrazvukový senzor nemal problém detegovať prekážku a zastaviť 10 cm pred ňou.

Tabuľka 3: Hodnoty : Ultrazvukový senzor – mimo vozovky

	Zastavilo?		
Poradie	Štíhla	Stredná	Široká
1	nie	nie	nie
2	nie	nie	nie
3	nie	nie	nie
4	nie	nie	nie
5	nie	nie	nie
Úspešnosť	100.00%	100.00%	100.00%

Podobne ako v predošlom teste, vozidlo nedetegovalo, že prekážka je v dráhe vozidla a správne vyhodnotilo, že nie je potrebné zastaviť vozidlo. Výsledky je možné vidieť v tabuľke 3.

Tabuľka 4 zobrazuje kritické zlyhanie systému, ktoré priamo vyplýva z jeho technického návrhu. Ultrazvukové senzory sú primárne určené na detekciu objektov na krátke vzdialenosti pri nízkych rýchlostiach (napr. parkovacie manévry) a nie sú vhodné na všeobecnú detekciu prekážok. Pri reálnych automobiloch je tento nedostatok vyrovnaný rozmiestnením senzorov po celom obvode vozidla.

Tabuľka 4: Hodnoty : Ultrazvukový senzor – pri vozovke

Poradie	Zastavilo?		
	Štíhla	Stredná	Široká
1	nie	nie	nie
2	nie	nie	nie
3	nie	nie	nie
4	nie	nie	nie
5	nie	nie	nie
Úspešnosť	0.00%	0.00%	0.00%

Použitý ultrazvukový modul HC-SR04 disponuje meracím uhlom 15° . Pri uplatnení sínusovej vety a stanovení brzdné vzdialenosti na 10 cm od senzora možno vypočítať, že efektívny priemer základne detekčného kužela dosahuje približne 2,6 cm. Pre pokrytie celej čelnej šírky vozidla (11 cm) by bolo nevyhnutné zvýšiť detekčnú vzdialenosť až na 42 cm. Takéto nastavenie by však spôsobovalo predčasné zastavenie vozidla a vnášalo do systému nežiaduce falošné detekcie.

8.3 Testovanie konvolučnej neurónovej siete

Tabuľka 5: Hodnoty : CNN – automobil

Poradie	Na vozovke		Pri vozovke
	Zastavilo?	Vzdialenosť	Obišlo?
1	áno	15 cm	
2	áno	12 cm	
3	áno	16 cm	
4	áno	13 cm	
5	áno	14 cm	
Úspešnosť	100.00%	14 cm	

Tabuľka 5 zobrazuje hodnoty testovania CNN YOLOv8 pre automobil. Úspešnosť zastavenia vozidla, keď sa prekážka nachádzala v strede vozovky je 100%. Priemerná vzdialenosť zastavenia je 14 cm.

V tabuľke 6 je možné vidieť výsledky testovania s nákladným autom. Dosahujeme rovnakú úspešnosť zastavenia ako v predošlej tabuľke – 100%. Priemerná vzdialenosť zastavenia je 16.4 cm, ďalej ako v prípade menšieho osobného automobilu.

Tabuľka 6: Hodnoty : CNN – Nákladné auto

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Obišlo?
1	áno	16 cm	
2	áno	18 cm	
3	áno	17 cm	
4	áno	16 cm	
5	áno	15 cm	
Úspešnosť	100.00%	16.4 cm	

Tabuľka 7: Hodnoty : CNN – Bicykel

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Obišlo?
1	áno	8 cm	
2	áno	9 cm	
3	áno	9 cm	
4	áno	8 cm	
5	áno	8 cm	
Úspešnosť	100.00%	8.4 cm	

Tabuľka 7 ukazuje výsledky testov pre bicykel. Systém vo všetkých testoch úspešne detekoval prekážku, s priemernou vzdialenosťou detekcie 8.4 cm – najmenej spomedzi všetkých testov.

Tabuľka 8: Hodnoty : CNN – Človek

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Obišlo?
1	áno	9 cm	
2	áno	9 cm	
3	áno	9 cm	
4	áno	10 cm	
5	áno	10 cm	
Úspešnosť	100.00%	9.4 cm	

V tabuľke 8 vidíme výsledky detekcie človeka na vozovke. Vozidlo vo všetkých testoch dokázalo zastaviť. Priemerná vzdialenosť zastavenia bola 9.4 cm.

Tabuľka 9: Hodnoty : CNN – Pes

	Na vozovke		Pri vozovke
Poradie	Zastavilo?	Vzdialenosť	Obišlo?
1	áno	8 cm	
2	áno	9 cm	
3	áno	10 cm	
4	áno	10 cm	
5	áno	9 cm	
Úspešnosť	100.00%	9.2 cm	

Tabuľka 9 zobrazuje výsledky testovania so psom ako prekážkou na vozovke. Úspešnosť zastavenie je 100%. Priemerná vzdialenosť zastavenia je 9.2 cm.

Záver

TODO

Literatúra

1. *2020 Autonomous Vehicle Technology Report* [online]. Wevolver, 2020. [cit. 2026-03-02]. Dostupné z: <https://www.wevolver.com/article/2020.autonomous.vehicle.technology.report>.
2. *J3016_202104: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles: Levels or categories of driving automation* [online]. Warrendale, PA, USA: SAE International, 2021 [cit. 2026-03-05]. Dostupné z: https://wiki.unece.org/download/attachments/128418539/SAE%20J3016_202104.pdf?api=v2.
3. PAREKH, D. et al. A Review on Autonomous Vehicles: Progress, Methods and Challenges. *Electronics* [online]. 2022 [cit. 2026-03-05]. ISSN 2079-9292. Dostupné z DOI: 10.3390/electronics11142162.
4. ANI KELKAR, K. H.; KELLNER, M. *Where to next? Insights from autonomous-vehicle experts* [online]. McKinsey & Company, 2026. [cit. 2026-03-04]. Dostupné z: <https://www.mckinsey.com/features/mckinsey-center-for-future-mobility/our-insights/future-of-autonomous-vehicles-industry>.
5. MARKUS, F. *The Great Sensor War Behind Self-Driving Cars Just Got Way More Interesting* [online]. Motortrend, 2026. [cit. 2026-03-04]. Dostupné z: <https://www.motortrend.com/news/latest-auto-lidar-radar-technology-ces-2026>.
6. SOARES, J. *NVIDIA Announces Alpamayo Family of Open-Source AI Models and Tools to Accelerate Safe, Reasoning-Based Autonomous Vehicle Development* [online]. Nvidia Newsroom, 2026. [cit. 2026-03-04]. Dostupné z: <https://nvidianews.nvidia.com/news/alpamayo-autonomous-vehicle-development>.
7. KOLLEWE, J. *Nvidia and UK Wealth Fund invest in British autonomous driving startup Oxa* [online]. The Guardian, 2026. [cit. 2026-03-04]. Dostupné z: <https://www.theguardian.com/technology/2026/mar/04/nvidia-uk-wealth-fund-invest-british-autonomous-driving-startup-oxa>.
8. *Waymo Slims Sensors to Drive Down Robotaxi Costs* [online]. Autonomous Vehicles & AI, 2026. [cit. 2026-03-04]. Dostupné z: <https://www.autonomous-vehicles-conference.com/news/waymo-slims-sensors-to-drive-down-robotaxi-costs>.
9. HAMZA, G.; ES-SADEK, M.; TAHER, Y. Artificial Intelligence In Self-Driving: Study of Advanced Current Applications [online]. 2023 [cit. 2026-03-08]. Dostupné z DOI: 10.31224/3209.

10. GRIGORESCU, S.; TRASNEA, B.; COCIAS, T.; MACESANU, G. A survey of deep learning techniques for autonomous driving. *Journal of Field Robotics* [online]. 2019, **37**, 362–386 [cit. 2026-03-08]. Dostupné z DOI: 10.1002/rob.21918.
11. BADUE, C. et al. Self-driving cars: A survey. *Expert Systems with Applications* [online]. 2021, **165**, 113816 [cit. 2026-03-08]. ISSN 0957-4174. Dostupné z DOI: <https://doi.org/10.1016/j.eswa.2020.113816>.
12. SINGH, K. *Tesla Launches Unsupervised Robotaxi Rides in Austin* [online]. Not a Tesla App, 2026. [cit. 2026-03-05]. Dostupné z: <https://www.notateslaapp.com/news/3527/tesla-launches-unsupervised-robotaxi-rides-in-austin>.
13. SRINIVAS, S.; RAMACHANDIRAN, S.; RAJENDRAN, S. Autonomous robot-driven deliveries: A review of recent developments and future directions. *Transportation Research Part E: Logistics and Transportation Review* [online]. 2022, **165** [cit. 2026-03-07]. ISSN 1366-5545. Dostupné z DOI: <https://doi.org/10.1016/j.tre.2022.102834>.
14. KOETSIER, J. *Starship's 2,700 Robots Have Made 9M Deliveries. Now They're Coming To Your City* [online]. Forbes, 2025. [cit. 2026-03-07]. Dostupné z: <https://www.forbes.com/sites/johnkoetsier/2025/10/15/starships-2700-robots-have-made-9m-deliveries-now-theyre-coming-to-your-city/>.
15. *Arduino® Nano ESP32* [SKU: ABX00083]. Arduino®, [b.r.]. Dostupné tiež z: <https://docs.arduino.cc/resources/datasheets/ABX00083-datasheet.pdf>.
16. *ESP32-CAM Development Board* [SKU: DFR602]. DFRobot, [b.r.]. Dostupné tiež z: https://media.digikey.com/pdf/Data%20Sheets/DFRobot%20PDFs/DFR0602_Web.pdf.
17. *OV2640 Color CMOS UXGA (2.0 MegaPixel) CAMERA CHIPTM with OmniPixel2TM Technology*. OmniVision, [b.r.]. Dostupné tiež z: https://www.uctronics.com/download/cam_module/OV2640DS.pdf.
18. *Ultrasonic Ranging Module HC - SR04*. ElecFreaks, [b.r.]. Dostupné tiež z: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>.
19. *DRV8833 Dual H-Bridge Motor Driver* [SKU: SLVSAR1E]. Texas Instruments, [b.r.]. Dostupné tiež z: <https://www.ti.com/lit/ds/symlink/drv8833.pdf?ts=1773296972820>.
20. STROUSTRUP, B. *C++ Programming Language* [online]. 4. vyd. Addison-Wesley, 2013 [cit. 2026-03-16]. ISBN 978-0-321-56384-2. Dostupné z: https://chenweixiang.github.io/docs/The_C++_Programming_Language_4th_Edition_Bjarne_Stroustrup.pdf.

21. KUHLMAN, D. *A Python Book: Beginning Python, Advanced Python, and Python Exercises* [online]. Platypus Global Media, 2011 [cit. 2026-03-16]. ISBN 978-0984221233. Dostupné z: https://web.archive.org/web/20120623165941/http://cutter.rexx.com/~dkuhlman/python_book_01.html.
22. KARI PULLI Anatoly Baksheev, K. K.; ERUHIMOV, V. Realtime Computer Vision with OpenCV. *Queue* [online]. 2023, **10**(4) [cit. 2026-03-16]. ISSN 1542-7730. Dostupné z DOI: 10.1145/2181796.
23. *Open Source Computer Vision Library* [online]. OpenCV, 2025. [cit. 2026-03-16]. Dostupné z: <https://github.com/opencv/opencv>.
24. HARRIS, C. R. et al. Array programming with NumPy. *Nature*. 2020, **585**, 357–362. Dostupné z DOI: 10.1038/s41586-020-2649-2.
25. VENKATESAN Ragav; Li, B. *Convolutional Neural Networks in Visual Computing: A Concise Guide* [online]. CRC Press, 2017 [cit. 2026-04-06]. ISBN 978-1-351-65032-8. Dostupné z: <https://books.google.sk/books?id=bAM7DwAAQBAJ>.
26. CIREGAN, D.; MEIER, U.; SCHMIDHUBER, J. Multi-column deep neural networks for image classification [online]. 2012 [cit. 2026-04-06]. Dostupné z DOI: 10.1109/CVPR.2012.6248110.
27. TERVEN, J.; CÓRDOVA-ESPARZA, D.-M.; ROMERO-GONZÁLEZ, J.-A. A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction* [online]. 2023, **5**(4), 1680–1716 [cit. 2026-03-16]. ISSN 2504-4990. Dostupné z DOI: 10.3390/make5040083.
28. JOCHER, G.; QIU, J.; CHAURASIA, A. *Ultralytics YOLO* [online]. 2023. [cit. 2026-03-16]. Dostupné z: <https://github.com/ultralytics/ultralytics>.

Použitie nástrojov umelej inteligencie

Google (2026), Gemini 3 Fast, celý text, úprava štylizácie textu.

OpenAI (2026), GPT-5.4, generovanie kódu programu.

Dodatok A: Výpis dlhého kódu

Zdrojový kód projektu je dostupný na:

<https://github.com/ruddpj/BP-AutonomousVehicle.git>.

Arduino programy

- Arduino/
 - ArduinoController/ArduinoController.ino
 - WifiCamera/WifiCamera.ino

Python programy

- Python/
 - connect.py
 - detect.py
 - interpret.py
 - main.py
 - remote.py
 - transform.py